

DEEPPFAKE VIDEO DETECTION ON CELEB-DF V2

Seven-Model Comparison with Low-Resolution and 64-Frame Evaluation

CAPSTONE PROJECT

Submitted in partial fulfillment of the requirements of the
Post Graduate Certification Program
in
Artificial Intelligence and Machine Learning

By

Varahagiri Venkata Sri Swati
(2025AIML002)

Aishwary Agrawal
(2025AIML017)

Devesh Srivastava
(2025AIML055)

Under the supervision of
(Tapas Chakraborty)

Project work carried out at
BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE
Pilani (Rajasthan) INDIA

October, 2026

PCAM ZC321 CAPSTONE PROJECT

DEEPFAKE VIDEO DETECTION ON CELEB-DF V2
Seven-Model Comparison with Low-Resolution and 64-Frame Evaluation

Submitted in partial fulfillment of the requirements of the
PGP - Artificial Intelligence and Machine Learning

By

Varahagiri Venkata Sri Swati
(2025AIML002)

Aishwary Agrawal
(2025AIML017)

Devesh Srivastava
(2025AIML055)

Under the supervision of
(Tapas Chakraborty)

Project work carried out at
BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE
PILANI (RAJASTHAN)

October, 2026

Acknowledgements

We would like to express our sincere gratitude to our mentor, Tapas Chakraborty, for the guidance, critical feedback and encouragement provided throughout this capstone project. The feedback received during the project reviews, in particular the suggestion to evaluate the final model on more frames and on low-resolution videos, directly shaped the last phase of this work.

We thank the faculty and the Work Integrated Learning Programmes Division of the Birla Institute of Technology and Science, Pilani, for designing the Post Graduate Program in Artificial Intelligence and Machine Learning and for the structured capstone framework within which this project was carried out.

We acknowledge the authors of the Celeb-DF v2 dataset for making their benchmark available for research, and the maintainers of the open-source libraries used in this work, including PyTorch, timm, InsightFace, ONNX Runtime, OpenCV and scikit-learn. GPU time was provided by Kaggle Notebooks and Google Colab, without which training and comparing seven deep-learning models would not have been possible.

Finally, we thank our families, colleagues and employers for their patience and support while we balanced this project with our professional responsibilities.

Varahagiri Venkata Sri Swati
Aishwary Agrawal
Devesh Srivastava

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE, PILANI

CERTIFICATE

This is to certify that the Capstone Project entitled **Deepfake Video Detection on Celeb-DF v2: Seven-Model Comparison with Low-Resolution and 64-Frame Evaluation** and submitted by **Varahagiri Venkata Sri Swati** (ID No. 2025AIML002) in partial fulfillment of the requirements of PCAM ZC321 Capstone Project, embodies the work done by the candidate under my supervision.

Place : Hyderabad

Signature of the Mentor

Date : 4/10/2026

Name : Tapas Chakraborty

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE, PILANI

CERTIFICATE

This is to certify that the Capstone Project entitled **Deepfake Video Detection on Celeb-DF v2: Seven-Model Comparison with Low-Resolution and 64-Frame Evaluation** and submitted by **Aishwary Agrawal** (ID No. 2025AIML017) in partial fulfillment of the requirements of PCAM ZC321 Capstone Project, embodies the work done by the candidate under my supervision.

Place : Hyderabad

Signature of the Mentor

Date : 4/10/2026

Name : Tapas Chakraborty

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE, PILANI

CERTIFICATE

This is to certify that the Capstone Project entitled **Deepfake Video Detection on Celeb-DF v2: Seven-Model Comparison with Low-Resolution and 64-Frame Evaluation** and submitted by **Devesh Srivastava** (ID No. 2025AIML055) in partial fulfillment of the requirements of PCAM ZC321 Capstone Project, embodies the work done by the candidate under my supervision.

Place : Hyderabad

Signature of the Mentor

Date : 4/10/2026

Name : Tapas Chakraborty

BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI
SECOND SEMESTER 2025-26
PCAM ZC321 CAPSTONE PROJECT

Project Title : Deepfake Video Detection on Celeb-DF v2: Seven-Model Comparison with Low-Resolution and 64-Frame Evaluation
Name of Mentor : Tapas Chakraborty
Name of Students : Varahagiri Venkata Sri Swati, Aishwary Agrawal, Devesh Srivastava
ID No. of Students : 2025AIML002, 2025AIML017, 2025AIML055

Abstract

Deepfake videos, in which one person’s face is replaced by another’s using deep generative models, are now realistic enough to deceive casual viewers and are increasingly misused for misinformation, fraud and impersonation. This capstone project builds and evaluates an end-to-end pipeline that takes a video file as input and returns one REAL or FAKE decision with a fake-probability score. It uses the Celeb-DF v2 benchmark (590 real celebrity videos, 300 real YouTube videos and 5,639 face-swapped videos; 6,529 in total), downloaded from Kaggle.

In the data-preparation step, videos were divided into training (4,385), validation (817) and holdout (809) sets with an identity-graph split, so that no person appears in more than one of these sets; the 518-video official test list was kept separate. Four face detectors were benchmarked, and RetinaFace was selected because it found every face at 50.2 FPS, about three times faster than MTCNN, and its five landmarks allow eye-aligned crops. Evenly spaced frames were sampled from every video, giving 73,429 aligned face images (99.95% yield, no video without a face).

In the team’s final Kaggle notebook, seven ImageNet-pretrained architectures (EfficientNet-B0, ResNet-50, Xception, ConvNeXt-Tiny, Swin-Tiny, EfficientNet-B4 and CLIP ViT-B/16) were fine-tuned under identical conditions on an NVIDIA T4 GPU. A video score is the mean of its face scores, and each model’s threshold was chosen on validation videos with Youden’s J statistic.

ConvNeXt-Tiny ranked first, with holdout AUC 0.9998 (bootstrap 95% CI 0.9993–1.0000), EER 0.0076, 98.27% accuracy (795 of 809 videos, no fake video missed) and an official-test AUC of 0.9990. The intervals of the six CNN and Swin models overlap, so these models are statistically close. Xception made the fewest holdout errors at its own threshold (8 of 809). ConvNeXt-Tiny was also the most robust model under JPEG compression, downscaling and blur (worst AUC drop 0.0010). The final Kaggle run measured about 643 classifier face images per second for ConvNeXt-Tiny. Grad-CAM shows attention on facial regions including the eyes, nose and mouth. CLIP ViT-B/16 remained the weakest model in the seven-model comparison (holdout AUC 0.9905).

Two resolution studies complete the work. An independent ConvNeXt-Tiny rebuild (Run B; holdout accuracy 97.28% and AUC 0.9998 at validation-selected threshold 0.4825) scored the three lowest-resolution real official-test videos on 64 frames before and after enhancement (CLAHE contrast enhancement and sharpening, with upscaling below 360 pixels): all 64 faces were detected for every condition and all three videos were classified REAL with a fake score of 0.0000 before and after enhancement. In Run A, a four-video check of high-resolution (974×500) and low-resolution (286×300) real and fake videos classified all four correctly. The report closes with recommendations and future work, including cross-dataset testing, video-level calibration and temporal models.

Key Words: Deepfake detection; Celeb-DF v2; ConvNeXt-Tiny; RetinaFace; transfer learning; video-level classification; identity-disjoint split; Grad-CAM; robustness; calibration; low-resolution video.

List of Symbols & Abbreviations Used

Symbol / Abbreviation	Meaning
AMP	Automatic Mixed Precision (FP16 training/inference)
AP	Average Precision (area under the precision-recall curve)
AUC	Area Under the ROC Curve
BCE	Binary Cross-Entropy loss
BGR / RGB	Blue-Green-Red (OpenCV) / Red-Green-Blue colour channel order
CI	Confidence Interval (95%, stratified bootstrap unless stated)
CLAHE	Contrast Limited Adaptive Histogram Equalisation
CLIP	Contrastive Language-Image Pre-training
CNN	Convolutional Neural Network
CUDA	Compute Unified Device Architecture (NVIDIA GPU platform)
ECE	Expected Calibration Error (15 bins)
EER	Equal Error Rate (point where FPR = FNR)
FF++	FaceForensics++ dataset
FN / FP	False Negative (fake predicted real) / False Positive (real predicted fake)
FNR / FPR	False Negative Rate / False Positive Rate
FPS	Frames Per Second
GAN	Generative Adversarial Network
GPU	Graphics Processing Unit
Grad-CAM	Gradient-weighted Class Activation Mapping
ID	Identity (a person appearing in the dataset)
MTCNN	Multi-Task Cascaded Convolutional Network (face detector)
ONNX / ORT	Open Neural Network Exchange / ONNX Runtime
ROC	Receiver Operating Characteristic
Run A	Seven-model comparison in the team’s final Kaggle notebook
Run B	Independent ConvNeXt-Tiny rebuild with the 64-frame low-resolution enhancement test
TN / TP	True Negative (real predicted real) / True Positive (fake predicted fake)
TPR / TNR	True Positive Rate (recall) / True Negative Rate (specificity)
ViT	Vision Transformer
p, P(fake)	Probability that a face (or video) is FAKE, output of the sigmoid
s(v)	Video-level fake score: mean of P(fake) over the faces of video v
t, τ	Decision threshold: a video is FAKE if $s(v) \geq t$
T	Temperature used to rescale logits in temperature scaling
J	Youden’s J statistic, $J = \text{TPR} - \text{FPR}$
w+	Positive-class weight in the BCE loss ($\text{pos_weight} = N_{\text{real}} / N_{\text{fake}}$)
$\sigma(\cdot)$	Sigmoid function

Symbol / Abbreviation	Meaning
N	Number of frames sampled from a video

List of Tables

Table 2.1: Summary of the reviewed literature.....	5
Table 3.1: Project team.....	7
Table 3.2: Hardware resources.....	8
Table 3.3: Software resources.....	8
Table 3.4: Data challenges, risks and mitigations.....	9
Table 3.5: Plan of work.....	10
Table 4.1: Composition of Celeb-DF v2.....	11
Table 4.2: Video-level split.....	12
Table 4.3: Frame sampling policy.....	13
Table 4.4: Face detector comparison.....	13
Table 4.5: Aligned face crops per split (Run A).....	15
Table 4.6: Training augmentation.....	15
Table 5.1: Seven candidate architectures.....	17
Table 5.2: Shared training settings.....	18
Table 5.3: Model-specific settings and training outcome (Run A, Kaggle T4).....	18
Table 5.4: Input options of the final Kaggle notebook.....	20
Table 5.5: Run B training history (ConvNeXt-Tiny, Kaggle T4).....	20
Table 5.6: Frame enhancement steps.....	22
Table 7.1: Measures used and their justification.....	40
Table 7.2: Seven-model comparison (Run A, final notebook).....	41
Table 7.3: Holdout confusion counts per model (105 real, 704 fake videos).....	44
Table 7.4: Holdout video AUC under degradation (up to 8 faces per video).....	45
Table 7.5: Calibration of the holdout video scores before and after temperature scaling.....	47
Table 7.6: Full-video demo results (Run A).....	48
Table 7.7: Speed measurements (Kaggle T4 GPU, Python timers).....	48
Table 7.8: Run B video-level metrics at threshold 0.4825.....	49
Table 7.9: Comparison of the two ConvNeXt-Tiny runs.....	51
Table 7.10: Videos selected for the enhancement test.....	53
Table 7.11: Original versus enhanced results (64 frames, threshold 0.4825).....	54
Table 7.12: High- versus low-resolution results (32 frames, threshold 0.0079).....	54
Table 7.13: Results against project objectives.....	57
Table A.1: Configuration values of the final notebook (Run A).....	64
Table B.1: Sections of the final notebook (Run A).....	65
Table B.2: Cells of the final Kaggle notebook (Run B).....	66
Table C.1: Output files (under the retinaface_models working folder).....	67

List of Figures

Figure 3.1: Consolidated machine-learning process flow (solution architecture).....	6
Figure 3.2: Video inference flow from input file to REAL / FAKE decision.....	7
Figure 4.1: Speed of the four face detectors on the same frames.....	13
Figure 4.2: Randomly sampled eye-aligned training crops: real (top) and fake (bottom).....	15
Figure 4.3: Videos (a) and aligned face crops (b) per split, by class.....	15
Figure 5.1: Run B training loss (a) and validation AUC (b); epoch 5 was selected.....	21
Figure 6.1: Kaggle screenshot of the final Kaggle notebook: environment preflight showing Tesla T4 and ONNX Runtime providers (5 October 2026).....	30
Figure 6.2: Kaggle screenshot of the final Kaggle notebook: manifest and leakage audit.....	31
Figure 6.3: Kaggle screenshot of the final Kaggle notebook: RetinaFace model download and the first face-cache shard.....	32
Figure 6.4: Kaggle screenshot of the final Kaggle notebook: remaining face-cache shards and crop counts.....	33
Figure 7.1: Holdout AUC with bootstrap 95% confidence intervals (a) and accuracy versus speed (b).....	42
Figure 7.2: Video-level ROC curves of the seven models on the holdout (left) and official test (right) videos, zoomed to $FPR \leq 0.3$	43
Figure 7.3: Holdout confusion matrices of the seven models, each at its own validation threshold	44
Figure 7.4: Training loss (a) and validation frame AUC (b) of the seven models in Run A.....	44
Figure 7.5: Worst holdout AUC drop per model under the four degradations (lower is better).....	46
Figure 7.6: Grad-CAM attention for one real (top) and one fake (bottom) holdout face, with each model's $P(\text{fake})$	48
Figure 7.7: Run B confusion matrices on the validation and holdout videos.....	50
Figure 7.8: Run A and Run B threshold-dependent error comparison; Run B metrics and confusion matrices.....	52
Figure 7.9: Extracted faces of the three low-resolution REAL videos, original (left) and enhanced (right).....	53
Figure 7.10: Aligned RetinaFace crops of the four videos of the resolution check (6 of 32 shown per video).....	56

Table of Contents

Chapter 1: Introduction.....	1
1.1 Background.....	1
1.2 Problem Statement.....	1
1.3 Objectives of the Project.....	1
1.4 Scope and Assumptions.....	2
1.5 Organisation of the Report.....	2
Chapter 2: Literature Review.....	3
2.1 Deepfake Datasets and Benchmarks.....	3
2.2 Frame-Level CNN Detectors.....	3
2.3 Artefact-Focused Methods.....	3
2.4 Temporal Methods.....	4
2.5 Vision Transformers and Foundation Models.....	4
2.6 Face Detection and Alignment.....	4
2.7 Explainability, Evaluation and Calibration.....	4
2.8 Summary and Research Gap.....	4
Chapter 3: Solution Architecture and Project Planning.....	6
3.1 Machine Learning Process Flow.....	6
3.2 Resources Needed for the Project.....	7
3.2.1 People.....	7
3.2.2 Hardware.....	8
3.2.3 Software.....	8
3.3 Potential Data Challenges and Risks.....	8
3.4 Detailed Plan of Work.....	9
Chapter 4: Data Pre-Processing.....	11
4.1 Dataset Description.....	11
4.2 Label Convention and Manifest Validation.....	11
4.3 Leak-Resistant Identity Split.....	11
4.4 Frame Sampling.....	12
4.5 Face Detector Selection.....	13
4.6 Face Detection, Alignment and Caching.....	14
4.7 Summary of the Prepared Data.....	15
4.8 Data Augmentation and Normalisation.....	15
4.9 Outlier Handling, Data Quality and Visualisation.....	16
Chapter 5: Machine Learning Modelling and Techniques.....	17
5.1 Problem Formulation.....	17
5.2 Candidate Architectures.....	17
5.3 Training Configuration.....	17
5.4 Training Procedure.....	18
5.5 Video-Level Aggregation and Threshold Selection.....	19

5.6 Evaluation Protocol of the Final Comparison.....	19
5.7 ConvNeXt-Tiny Rebuild (Run B, Run B).....	20
5.8 Low-Resolution 64-Frame Enhancement Test (Run B).....	21
5.9 High- versus Low-Resolution Check (Run A).....	22
Chapter 6: Code and Screenshots.....	23
6.1 Final Team Notebook (Run A).....	23
6.1.1 Environment, Data and Model Registry.....	23
6.1.2 Training and Evaluation of the Seven Models.....	24
6.1.3 Comparison, Robustness and Calibration.....	26
6.1.4 Demo, Export and High / Low Resolution Check.....	27
6.2 Low-Resolution Run B (Run B).....	29
6.2.1 Environment and GPU Check.....	29
6.2.2 Manifest and Leakage Audit.....	30
6.2.3 RetinaFace Detection and Eye Alignment.....	31
6.2.4 Model Construction and Training Loop.....	34
6.2.5 Threshold Selection and Evaluation.....	36
6.2.6 Enhancement and 64-Frame Video Inference.....	38
Chapter 7: Results and Interpretation.....	40
7.1 Justification of Measures and Metrics.....	40
7.2 Seven-Model Comparison (Run A).....	40
7.3 Official Test Benchmark.....	45
7.4 Robustness to Compression, Downscaling and Blur.....	45
7.5 Probability Calibration.....	46
7.6 Explainability with Grad-CAM.....	48
7.7 End-to-End Demo, Speed and Export.....	48
7.8 ConvNeXt-Tiny Rebuild (Run B) and Comparison of the Two Runs.....	49
7.9 Low-Resolution 64-Frame Enhancement Results (Run B).....	53
7.10 High- versus Low-Resolution Check (Run A).....	54
7.11 Summary of Results Against Objectives.....	57
Chapter 8: Conclusions and Recommendations.....	58
8.1 Conclusions.....	58
8.2 Recommendations.....	58
Chapter 9: Directions for Future Work.....	60
References.....	69
Appendices.....	69
Appendix A: Configuration of the Final Notebook.....	64
Appendix B: Notebook Maps.....	65
Appendix C: Files Produced by the Pipeline.....	67
List of Publications / Conference Presentations.....	55
Checklist of Items for the Final Report.....	56

Chapter 1: Introduction

1.1 Background

A deepfake is a synthetic video in which the face of one person is replaced by, or re-animated to look like, the face of another person using deep generative models such as autoencoders and generative adversarial networks [1]. Face-swap tools are now freely available, and the quality of their output has improved to the point where many fake videos can no longer be recognised by a casual viewer [2, 3]. Such videos can be misused for political misinformation, financial fraud, impersonation and non-consensual content, and their existence also makes it easier to dismiss genuine recordings as fake.

Automatic detection is therefore an important applied machine-learning problem. Public benchmarks such as FaceForensics++ [4], Celeb-DF [5] and the DeepFake Detection Challenge (DFDC) dataset [6] have made it possible to train and compare detectors. Celeb-DF v2 was designed to be harder than earlier datasets: its fake videos were produced with an improved synthesis process that reduces visible artefacts such as colour mismatch and blending boundaries [5]. This project uses Celeb-DF v2 as its benchmark.

Most practical detectors work on faces rather than on complete frames. The face is located, cropped and passed to an image classifier, and the per-face scores are combined into one decision for the video. This project follows that design and concentrates on practical questions: which face detector and which classifier backbone work best, how reliable and well calibrated the result is on people never seen during training and on degraded video, how fast the complete pipeline is, and how the detector behaves on low-resolution input.

1.2 Problem Statement

Given a video that contains a human face, automatically decide whether the video is **REAL** or **FAKE** (face-swapped) and report a fake-probability score for the whole video. To be useful in practice, the detector should:

- generalise to people who were not seen during training, which requires an evaluation split with no identity overlap between training and test data;
- remain reliable when the video is compressed, downscaled, blurred or of low resolution, as happens when videos are re-uploaded to social media;
- process a complete video within a few seconds on a single GPU;
- show which face regions influenced its decision, so that a human reviewer can check the result.

1.3 Objectives of the Project

- O1 Build a reproducible data pipeline that turns the raw Celeb-DF v2 videos into aligned face images, using an identity-disjoint train / validation / holdout split.

- O2 Select a face detector by benchmarking OpenCV Haar, MTCNN, RetinaFace and YOLOv8 on the same frames.
- O3 Fine-tune and compare seven ImageNet-pretrained architectures under identical conditions and select the best one by holdout video-level AUC.
- O4 Convert face scores into video decisions with a threshold chosen on validation data, and evaluate on the holdout split and the official Celeb-DF test list using AUC with bootstrap confidence intervals, accuracy, precision, recall, F1, EER and confusion matrices.
- O5 Test robustness to JPEG compression, downscaling and blur; check probability calibration; explain predictions with Grad-CAM; measure speed; and demonstrate and export the complete video-to-decision pipeline.
- O6 Evaluate the final model on low-resolution input: 64 frames per video with and without frame enhancement, and a comparison of high- and low-resolution real and fake videos.

1.4 Scope and Assumptions

- **Data:** Celeb-DF v2 only (6,529 videos), obtained from a Kaggle mirror [7]; the manipulation type is face swapping.
- **Faces:** one dominant face per frame is assumed; when several faces are detected, the largest one is used.
- **Models:** frame-level image classifiers; the video score is the mean of the face scores. Temporal models and audio are outside the scope of this project.
- **Hardware:** a single NVIDIA Tesla T4 GPU in Kaggle Notebooks.
- Experiments reported. Run A is the seven-model comparison in the team’s final Kaggle notebook (deepfake-detection-final.ipynb), which also contains the robustness, calibration, Grad-CAM, demo and export steps and the four-video high- versus low-resolution check. Run B is a separate rebuild of the selected ConvNeXt-Tiny pipeline (Run B workflow) that adds the 64-frame low-resolution enhancement test. An earlier version of the comparison, run on Google Colab and shown in the project presentation, is superseded by Run A.

1.5 Organisation of the Report

Chapter 2 reviews previous work on deepfake datasets, detectors, face detection, explainability and calibration. Chapter 3 presents the machine-learning process flow, the resources used, the data challenges and risks, and the plan of work. Chapter 4 describes the data pre-processing steps, and Chapter 5 the modelling and evaluation techniques, including the two resolution studies. Chapter 6 contains the key code and notebook outputs. Chapter 7 justifies the metrics used and interprets all results. Chapters 8 and 9 give the conclusions, recommendations and directions for future work, followed by the references, appendices and the completed checklist.

Chapter 2: Literature Review

This chapter summarises the previous work that informed the design choices of the project. It is organised by topic: datasets, frame-level detectors, artefact-focused methods, temporal methods, transformers and foundation models, face detection, and explainability and evaluation.

2.1 Deepfake Datasets and Benchmarks

FaceForensics++ [4] contains 1,000 original YouTube videos manipulated with four methods (DeepFakes, Face2Face, FaceSwap and NeuralTextures) at several compression levels. Its benchmark showed that an Xception network fine-tuned on face crops is a strong detector, and that accuracy falls noticeably on heavily compressed video. **Celeb-DF v2** [5] contains 590 real interview videos of 59 celebrities, 300 additional real YouTube videos and 5,639 face-swapped videos produced with an improved synthesis method; it also defines an official test list of 518 videos. Its authors showed that many detectors that performed well on earlier datasets performed much worse on Celeb-DF. The **DFDC** dataset [6] is larger still, with more than 100,000 clips recorded with paid actors, and its public competition showed that strong ImageNet-pretrained CNN backbones applied to face crops were a common basis of the best entries.

2.2 Frame-Level CNN Detectors

Early work such as **MesoNet** [8] used compact CNNs that focus on mid-level (mesoscopic) image detail. Later work relied on transfer learning from large ImageNet-pretrained backbones [9]. **Xception** [10], built from depthwise separable convolutions, became the standard deepfake baseline after FaceForensics++ [4]. **ResNet** [11] introduced residual connections and is a common reference backbone. **EfficientNet** [12] scales depth, width and resolution together and gives good accuracy per parameter; EfficientNet-B0 served as this project’s mid-term baseline, and the EfficientNet-B4 weights used here come from Noisy Student self-training [13]. **ConvNeXt** [14] modernises the ResNet design with ideas taken from vision transformers (larger depthwise kernels, LayerNorm, GELU activations and inverted bottlenecks) and matches transformer accuracy while remaining a pure CNN.

2.3 Artefact-Focused Methods

Several methods target the traces left by the face-swap process. Li and Lyu [15] showed that the warping needed to fit a synthesised face to the target leaves resolution inconsistencies that a CNN can detect. **Face X-ray** [16] predicts the blending boundary between the inserted face and the background, which makes the detector less dependent on a particular generator. **Multi-attentional** detection [17] uses several attention maps to focus on local texture differences. A common message of these works is that the strongest evidence lies around the blended regions of the face, such as the eyes, nose, mouth and face boundary; the Grad-CAM results in Chapter 7 are consistent with this.

2.4 Temporal Methods

Video adds a time dimension that single-frame models ignore. Güera and Delp [18] combined a CNN feature extractor with an LSTM to detect frame-to-frame inconsistencies, and **LipForensics** [19] used mouth-movement features to obtain detectors that generalise better and are more robust to compression. Temporal models need more compute and more complex data loading, so this project uses frame-level classification with score averaging and lists temporal modelling as future work.

2.5 Vision Transformers and Foundation Models

The **Vision Transformer (ViT)** [20] treats an image as a sequence of patches processed by self-attention. **Swin Transformer** [21] restricts attention to shifted local windows, which makes it efficient for dense image detail. **CLIP** [22] learns image and text encoders jointly from a very large set of image-text pairs; Ojha et al. [23] showed that CLIP image features generalise well for detecting images from unseen generators. These results motivated the inclusion of Swin-Tiny and CLIP ViT-B/16 in the comparison, to test whether attention-based and foundation models outperform CNNs on face-swap detection.

2.6 Face Detection and Alignment

The quality of face crops directly affects any face-based detector. The **Viola-Jones** Haar cascade [24] is fast on a CPU but less reliable for rotated or partly occluded faces. **MTCNN** [25] uses a three-stage CNN cascade and returns five facial landmarks. **RetinaFace** [26] is a single-stage detector that predicts boxes and five landmarks at several scales and is widely available through the InsightFace project [27]. **YOLOv8** [28] is a fast general-purpose detector that, as used here, returns boxes without landmarks. Landmarks matter because they allow every face to be rotated and scaled so that the eyes are at fixed positions, which gives the classifier consistent input.

2.7 Explainability, Evaluation and Calibration

Grad-CAM [29] produces a heat map of the image regions that most influenced a CNN’s output by weighting the last convolutional feature maps with the gradients of the class score. For evaluation, the ROC curve and its area (AUC) [30] measure ranking quality independently of any threshold and are the standard metrics on imbalanced deepfake benchmarks; bootstrap resampling [31] gives confidence intervals for such metrics. **Youden’s J statistic** [32], $J = \text{TPR} - \text{FPR}$, gives a simple rule for choosing a threshold on a validation set. Guo et al. [33] showed that modern neural networks are often over-confident and proposed **temperature scaling**, which divides the logits by a single fitted temperature to improve calibration without changing the ranking.

2.8 Summary and Research Gap

Table 2.1 summarises the reviewed work and its relevance to this project.

Table 2.1: Summary of the reviewed literature

Work	Approach / key idea	Use in this project
Rössler et al. [4]	FF++ benchmark; Xception on face crops	Xception baseline; effect of compression
Li et al. [5]	Celeb-DF v2 with improved synthesis	Dataset and official test list used here
Afchar et al. [8]	Compact CNN (MesoNet)	Early frame-level detector
Tan and Le [12]; Xie et al. [13]	Compound-scaled CNN; Noisy Student weights	EfficientNet-B0 (baseline) and B4 compared
Liu et al. [14]	Modernised CNN (ConvNeXt)	Selected final model
Liu et al. [21]; Radford et al. [22]	Window attention; image-text pre-training	Swin-Tiny and CLIP ViT-B/16 compared
Li et al. [16]; Zhao et al. [17]	Blending boundary; multi-attention	Explains why face-region cues matter
Güera and Delp [18]; Haliassos et al. [19]	Temporal CNN-LSTM; lip movements	Future work
Deng et al. [26]; Zhang et al. [25]	Landmark-based face detectors	RetinaFace selected
Selvaraju et al. [29]	Gradient-based class activation maps	Model explanation
Guo et al. [33]	Temperature scaling for calibration	Calibration check of all seven models

The review shows four gaps that this project addresses. First, published comparisons often use different pre-processing for different models, so differences in results may come from the data pipeline rather than the architecture; here all seven models share the same face images, split, augmentation and training budget. Second, random video-level splits can place the same person in training and test data and inflate results; here train, validation and holdout sets are identity-disjoint. Third, robustness, calibration and low-resolution behaviour are often not reported; here they are tested explicitly. Fourth, end-to-end speed from video file to decision is rarely measured; here it is.

Chapter 3: Solution Architecture and Project Planning

3.1 Machine Learning Process Flow

The consolidated approach is shown in Figure 3.1. It consists of three steps, each producing a saved output that the next step reuses, so that expensive work (face extraction and training) is done once.

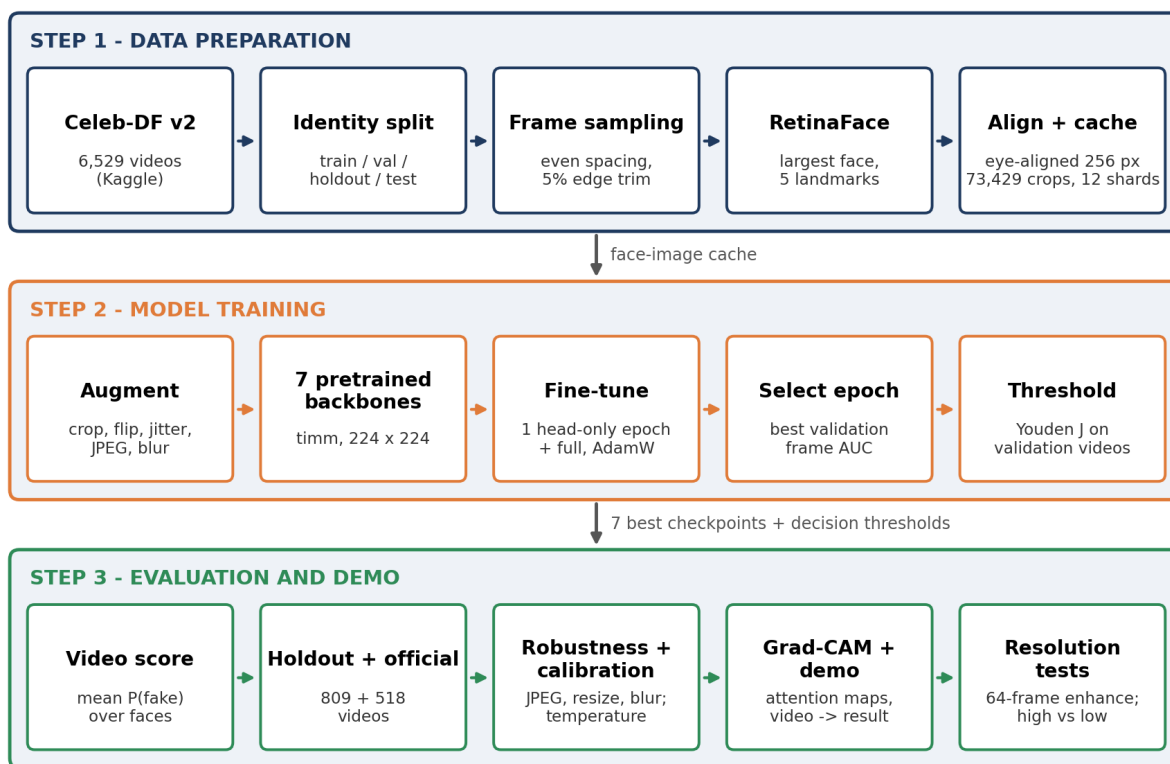


Figure 3.1: Consolidated machine-learning process flow (solution architecture)

1. **Data acquisition:** the Celeb-DF v2 dataset attached in Kaggle is located automatically, and the folder counts (590 / 300 / 5,639) are verified.
2. **Manifest and split:** every video is parsed into identities and labels, the official test list is separated, and the remaining videos are split into identity-disjoint train, validation and holdout sets.
3. **Frame sampling:** evenly spaced frames are taken from each video, skipping the first and last 5%.
4. **Face detection and alignment:** RetinaFace finds the largest face and its five landmarks; the face is rotated and scaled so that the eyes are horizontal and at fixed positions in a 256×256 crop.
5. **Caching:** crops are saved as JPEG (quality 95) and packed into 12 shards, so that every model trains on the same stored faces instead of decoding videos again.

6. **Training:** each pretrained backbone is fine-tuned on the cached faces with the same augmentation and epoch budget.
7. **Model selection and threshold:** the epoch with the best validation frame AUC is kept, and the video threshold is chosen on validation video scores with Youden's J.
8. **Evaluation:** holdout and official test videos are scored and compared with their labels; bootstrap intervals, robustness, calibration, Grad-CAM and speed are measured.
9. **Inference and export:** a new video passes through the same steps and receives a REAL / FAKE decision; the best model, its threshold and its pre-processing settings are saved as one inference bundle.
10. **Resolution tests:** the selected model is tested on 64 frames per video before and after enhancement (Run B) and on high- and low-resolution real and fake videos (Run A).

Figure 3.2 shows the inference path used for any single video, including the demo and both resolution tests.

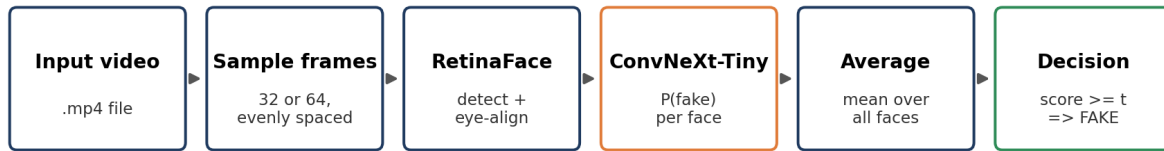


Figure 3.2: Video inference flow from input file to REAL / FAKE decision

3.2 Resources Needed for the Project

3.2.1 People

Table 3.1: Project team

Role	Name	BITS ID No.
Student	Varahagiri Venkata Sri Swati	2025AIML002
Student	Aishwary Agrawal	2025AIML017
Student	Devesh Srivastava	2025AIML055
Mentor	Tapas Chakraborty	-

The team carried out the data preparation, model training, evaluation and documentation, with guidance and review from the mentor. The final notebook is organised in four parts: data and face extraction; training components and training; comparison, robustness, calibration, explainability and demo; and the high- versus low-resolution check.

3.2.2 Hardware

Table 3.2: Hardware resources

Resource	Specification	Used for
Kaggle GPU notebook	2 × NVIDIA Tesla T4 (15,360 MiB each; the pipeline uses one), driver 580.178.04	Run A: face cache, training of the seven models (21-48 min each), evaluation, robustness, calibration, Grad-CAM, demo, export, resolution check. Run B: ConvNeXt-Tiny rebuild and 64-frame test
Google Colab GPU runtime	NVIDIA Tesla T4, 16 GB	Earlier version of the seven-model comparison shown in the project presentation (superseded by Run A)
Cloud storage	/kaggle/working (Kaggle notebooks)	Manifest, face-cache shards, checkpoints, predictions, metrics, demo videos, inference bundle
Personal laptops	Standard office laptops	Development, analysis, presentation and report writing

3.2.3 Software

Table 3.3: Software resources

Package / tool	Version	Purpose
Python	3.12 (Kaggle image)	Programming language
PyTorch [34] / torchvision	2.11.0 (Run A), 2.10.0 (Run B); CUDA 12.8	Model training, inference, transforms
timm [35]	1.0.11	Pretrained backbones (e.g. convnext_tiny.fb_in22k_ft_in1k)
InsightFace [27]	0.7.3 (buffalo_l pack)	RetinaFace detector (det_10g.onnx)
ONNX Runtime GPU	1.26.0	Runs RetinaFace on the GPU (CUDA execution provider)
OpenCV	Kaggle default	Video decoding, alignment, enhancement (CLAHE)
scikit-learn [36]	Kaggle default	AUC, ROC, confusion matrix, precision / recall / F1, log loss
SciPy	Kaggle default	Logit / sigmoid functions for temperature scaling
pandas, NumPy, Matplotlib	Kaggle default	Manifests, metrics tables, plots
Kagglehub	Latest	Dataset download fallback (Run B, Option 1)
Kaggle Notebooks / Google Colab	-	GPU notebook environments

3.3 Potential Data Challenges and Risks

Table 3.4 lists the main data challenges and project risks identified at the start of the work, and how each was handled.

Table 3.4: Data challenges, risks and mitigations

#	Challenge / risk	Impact	Mitigation used
1	Class imbalance: 890 real vs 5,639 fake videos	Model biased towards FAKE; accuracy misleading	More frames from real videos (32 vs 8); loss weight $w_+ = 0.508$; AUC as main metric; threshold tuned on validation
2	Identity leakage between splits	Over-optimistic results	Identity-graph split with automatic leakage audit
3	Official test list uses 1 = real, 0 = fake	Silently inverted labels	Labels flipped and cross-checked against folders; run stops on any mismatch
4	Missed faces, several faces in a frame	Missing or wrong crops	RetinaFace (score ≥ 0.5), largest face kept, frames without a face skipped (99.95% yield)
5	Train / inference pre-processing mismatch	Lower accuracy in deployment	Same detector, alignment and BGR $\rightarrow$ RGB conversion in training and inference; settings saved in the bundle
6	Compression, blur and low resolution	Accuracy drop on re-uploaded videos	JPEG and blur augmentation; robustness test; two low-resolution tests
7	GPU session limits and disconnects	Lost training progress	12 cache shards; last.pt saved after every epoch with automatic resume; finished runs skipped
8	Large raw video volume	Slow repeated decoding	Faces extracted once and cached
9	Unseen manipulation methods	Possible drop on other generators	Stated as a limitation; cross-dataset testing proposed as future work
10	Ethical use of face data	Misuse or over-trust of predictions	Research dataset only; outputs reported as probabilities; human review recommended

3.4 Detailed Plan of Work

The work was carried out in nine phases (Table 3.5). Each phase produced a saved output used by the next phase, which made it possible to resume after interruptions and to repeat parts of the work on a different platform.

Table 3.5: Plan of work

Phase	Activity	Main tasks	Output
1	Problem definition and literature review	Scope, success metrics, candidate methods	Problem statement, approach
2	Data acquisition and validation	Obtain Celeb-DF v2 from Kaggle; verify folder counts; parse official test list	Validated video manifest
3	Leak-resistant split	Identity graph, greedy split, leakage audit	videos.csv with split column
4	Face-detector benchmark	Haar, MTCNN, RetinaFace, YOLOv8 on 10 videos $\times$ 15 frames	RetinaFace selected
5	Face extraction and caching	Sampling, detection, eye alignment, JPEG cache in 12 shards	73,429 aligned face images
6	Baseline and seven-model training	EfficientNet-B0 baseline (mid-term), first comparison on Colab, final comparison on Kaggle (Run A)	7 best checkpoints and thresholds
7	Evaluation (Run A)	Holdout and official test, bootstrap intervals, confusion matrices, ROC, robustness, calibration, Grad-CAM, demo, speed, export	Model comparison; ConvNeXt-Tiny selected
8	Resolution studies	Run B: ConvNeXt-Tiny rebuild with 64-frame enhancement test; Run A: four-video high vs low resolution check	Low-resolution results
9	Documentation	Presentation and project report	Final deliverables

All nine phases have been completed.

Chapter 4: Data Pre-Processing

This chapter describes how the raw videos were turned into a clean, labelled and leak-free set of aligned face images. All steps are implemented in the project notebooks and are deterministic for a given seed (42).

4.1 Dataset Description

Celeb-DF v2 [5] was obtained from Kaggle [7]. Its composition, as counted by the notebook, is shown in Table 4.1.

Table 4.1: Composition of Celeb-DF v2

Folder / file	Content	Label	Videos
Celeb-real	Real interview videos of celebrities (file name id<a>_<n>)	REAL (0)	590
YouTube-real	Additional real YouTube videos (numeric file name)	REAL (0)	300
Celeb-synthesis	Face-swapped videos combining identities a and b (id<a>_id_<n>)	FAKE (1)	5,639
Total			6,529
List_of_testing_videos.txt	Official test list (178 real, 340 fake), kept separate	both	518

The dataset is strongly imbalanced at video level: real videos are only 13.6% of the total (890 of 6,529). This imbalance shapes several later choices, including frame sampling, the loss weight and the choice of AUC as the main metric.

4.2 Label Convention and Manifest Validation

A manifest (videos.csv) with one row per video is built before any image processing. The following checks are applied automatically, and the notebook stops if any of them fails:

- The label convention REAL = 0, FAKE = 1 is used throughout. The official test file uses the opposite convention (1 = real, 0 = fake), so its flags are inverted when read.
- Every video named in the official test list must exist in the dataset folders, and its inverted label must agree with the label implied by its folder.
- Every file name must match one of three patterns (Celeb-real, YouTube-real or fake); any unrecognised name raises an error, so no video is silently dropped.
- The folder counts must match the expected values (590 / 300 / 5,639).

4.3 Leak-Resistant Identity Split

If the same person appeared in both training and test data, the model could learn to recognise the person instead of the manipulation, and test results would be too optimistic. The split therefore works on identities rather than on videos:

1. Each video is parsed into one identity (real video) or two identities (fake video, $\text{id}_{\langle a \rangle_id_{\langle b \rangle}}$).
2. A union-find structure joins the two identities of every fake video, so that people who appear together in any fake video form one connected component.
3. The official test videos are set aside. The remaining components are sorted by size and assigned greedily to the split that is furthest below its target share (75% train, 15% validation, 10% holdout); components with fake videos and real-only components are assigned separately.
4. A leakage audit checks that train, validation and holdout share no identity.

The audit passed. Because whole components are assigned, the final shares differ from the targets: 73.0% train, 13.6% validation and 13.5% holdout of the 6,011 non-test videos (Table 4.2). The audit also reports that the official test list shares 36 identities with the training set, which is expected by the design of Celeb-DF.

Table 4.2: Video-level split

Split	Real	of which YouTube	Fake	Total	Fake per real	Purpose
Train	495	171	3,890	4,385	7.86	Model fitting
Validation	112	35	705	817	6.29	Epoch selection and threshold
Holdout	105	24	704	809	6.70	Main unseen-identity result
Official test	178	70	340	518	1.91	Celeb-DF benchmark result
Total	890	300	5,639	6,529	6.34	

Because the official test list shares identities with training, the holdout split is treated as the main measure of generalisation to unseen people, and the official test list as the published benchmark.

4.4 Frame Sampling

For a video with N frames, the sampler skips the first and last 5% (to avoid fades and titles) and takes n evenly spaced frame indices between $\text{first} = \lfloor 0.05N \rfloor$ and $\text{last} = \lfloor 0.95N \rfloor - 1$. If fewer than half of the requested frames can be read by seeking, the video is decoded sequentially instead. Different numbers of frames are taken from real and fake videos to reduce the class imbalance at face level (Table 4.3).

Table 4.3: Frame sampling policy

Video type	Frames	Reason
Real video (train / val / holdout)	32	Real videos are rare, so more faces are taken from each
Fake video (train / val / holdout)	8	Fake videos are plentiful; fewer faces per video suffice
Official test video	16	Same number for both classes, no rebalancing
Demo and high / low resolution check (Run A)	32	End-to-end prediction on full videos
Low-resolution enhancement test (Run B)	64	More coverage of each video

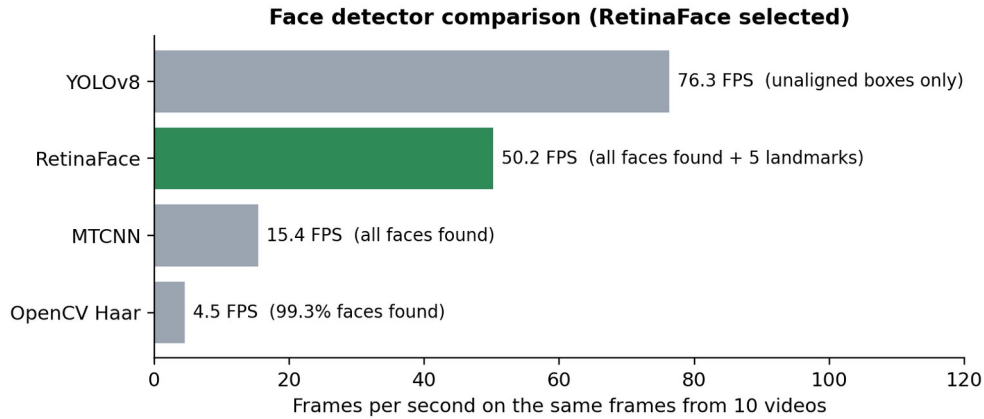
With this policy the training set has 15,805 real and 31,120 fake faces, a ratio of about 1 : 2, compared with 1 : 7.9 at video level.

4.5 Face Detector Selection

Four detectors were compared on the same 15 frames from each of 10 videos on a T4 GPU (Table 4.4 and Figure 4.1). Only detection was compared; the classifier was not involved.

Table 4.4: Face detector comparison

Detector	Speed (FPS)	Face yield	Landmarks	Decision
OpenCV Haar cascade [24]	4.5	99.3%	No	
MTCNN [25]	15.4	100%	Yes (5)	
RetinaFace, buffalo_l [26]	50.2	100%	Yes (5)	Selected
YOLOv8n [28]	76.3	100%	No (boxes only)	

**Figure 4.1:** Speed of the four face detectors on the same frames

RetinaFace was selected: it found every face, it was about three times faster than MTCNN with the same face yield, and its five landmarks allow eye-aligned crops. YOLOv8 was faster but returns unaligned boxes. RetinaFace is also used at inference time, because changing the detector would create a mismatch between training and inference crops.

4.6 Face Detection, Alignment and Caching

Each sampled frame is processed as follows:

1. RetinaFace (InsightFace buffalo_l, det_10g model, input size 640×640 , minimum score 0.50) is run through ONNX Runtime on the GPU; the notebook stops if the CUDA provider is not active.
2. If several faces are found, the one with the largest bounding box is kept; frames without a face are skipped.
3. The two eye landmarks define the rotation angle (atan2 of the eye vector) and the scale, so that the eye distance becomes $(1 - 2 \times 0.35) \times 256 \approx 77$ pixels and the eye line lies at 35% of the crop height.
4. An affine warp (bicubic interpolation, replicated borders) produces a 256×256 crop; a crop is rejected if the eye distance is almost zero.
5. Crops are saved as JPEG (quality 95) in split / class folders and packed into 12 tar shards; on a restart the shards are restored instead of being recomputed.

Figure 4.2 shows randomly sampled aligned training crops. The alignment places the eyes at the same position in every crop, so that the classifier can compare like with like.

deepfake-video-detection-final-capstone-run-a-r... Draft saved

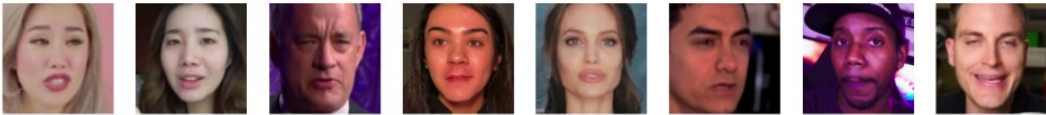
File Edit View Run Settings Add-ons Help

+ [24] label real fake split

	label	real	fake
train	15805	31120	
val	3584	5640	
holdout	3360	5632	
test	2848	5440	

```
[25]: samples = SPLIT_FRAMES["train"].groupby("label").sample(8, random_state=CFG.split_seed)
for label, group in samples.groupby("label"):
    show_images([cv2.imread(path) for path in group.path], f'Train crops: {CLASS_NAMES[label]}")
```

Train crops: real



Train crops: fake



+ Code + Markdown

4. Training components — Participant 2

- Frame classifier with one logit and `BCEWithLogitsLoss(pos_weight = real / fake)` on the training frames.
- Epoch 0 trains only the zero-initialised head; later epochs fine-tune the backbone (CLIP: last 4 blocks).
- AdamW with separate head / backbone learning rates and no decay on norms and biases, 5% warmup then cosine, fp16 autocast, gradient clipping.
- The best epoch is chosen on validation frame AUC with early stopping. The decision threshold is picked on validation (Youden's J) and reused unchanged for holdout and test.

Figure 4.2: Randomly sampled eye-aligned training crops: real (top) and fake (bottom)

4.7 Summary of the Prepared Data

The preprocessing completed for all 6,529 videos with no video left without a face. The final notebook produced 73,429 indexed aligned face images. Table 4.5 gives the Run A counts, and Figure 4.3 visualises the video and face counts per split.

Table 4.5: Aligned face crops per split (Run A)

Split	Real faces	Fake faces	Total
Train	15,805	31,120	46,925
Validation	3,584	5,640	9,224
Holdout	3,360	5,632	8,992
Official test	2,848	5,440	8,288
Total	25,597	47,832	73,429

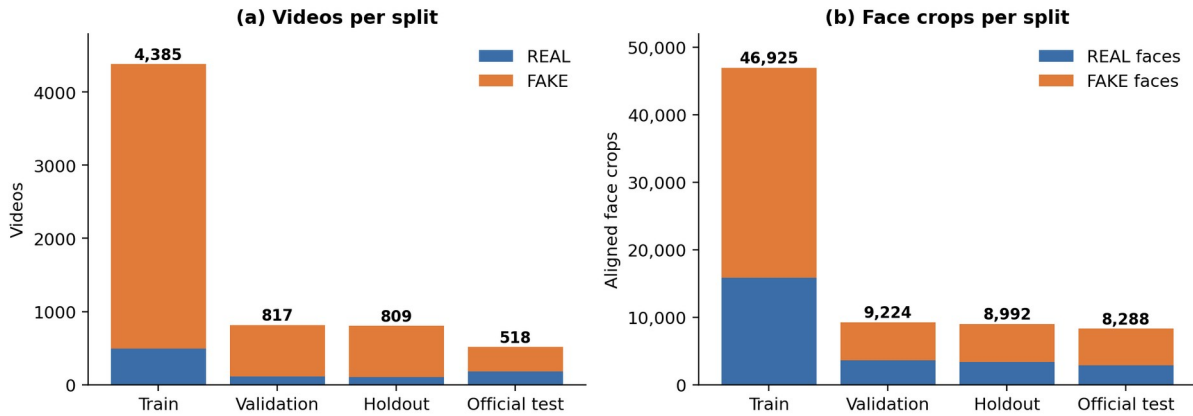


Figure 4.3: Videos (a) and aligned face crops (b) per split, by class

4.8 Data Augmentation and Normalisation

Training images are augmented on the fly to make the model less sensitive to framing, colour and compression (Table 4.6). Validation, holdout and test images are only resized to 224×224 . Every model is normalised with the mean and standard deviation of its own pretrained weights.

Table 4.6: Training augmentation

Transform	Setting	Purpose
RandomResizedCrop	224×224 , scale 0.8-1.0, aspect 0.95-1.05	Small shifts and zoom
RandomHorizontalFlip	$p = 0.5$	Left / right symmetry
ColorJitter	brightness 0.2, contrast 0.2, saturation 0.15	Lighting and camera differences
RandomJPEG	quality 40-95, $p = 0.5$	Compression as in re-uploaded videos
GaussianBlur	kernel 3, σ 0.1-1.2, $p = 0.1$	Slight defocus or motion
Normalize	per-model mean / std (timm data config)	Match pretraining statistics

4.9 Outlier Handling, Data Quality and Visualisation

- **Detection outliers:** low-confidence detections (score < 0.5) are ignored; background faces are removed by keeping the largest face; crops with degenerate eye landmarks are rejected.
- **Unreadable frames:** frames that cannot be decoded are skipped, with a sequential-decoding fallback; videos that cannot be opened are excluded from the resolution tests.
- **Consistency checks:** every cached crop must map to exactly one manifest row, and its folder split must match the manifest split; otherwise the run stops.
- **Safe cache restore:** archive members are extracted with a safety filter so that no file can be written outside the cache folder.
- **Visual summaries:** sample crops for one real and one fake video, random training crops (Figure 4.2), the split counts (Figure 4.3) and the crops of the resolution tests (Chapter 7) were inspected to confirm that alignment and labelling were correct.

Chapter 5: Machine Learning Modelling and Techniques

5.1 Problem Formulation

Each aligned face x is classified by a network f that outputs one logit $z = f(x)$. The probability that the face is fake is $p = \sigma(z)$. Training minimises the class-weighted binary cross-entropy

$$L = - [w^+ \cdot y \cdot \log \sigma(z) + (1 - y) \cdot \log(1 - \sigma(z))],$$

where $y = 1$ for FAKE and $w^+ = N_{\text{real}} / N_{\text{fake}} = 15,805 / 31,120 = 0.508$ (PyTorch BCEWithLogitsLoss with pos_weight). Because fake faces are still about twice as common as real ones in training, $w^+ < 1$ down-weights them so that both classes contribute equally to the loss.

5.2 Candidate Architectures

Seven ImageNet-pretrained [9] backbones from the timm library [35] were compared (Table 5.1). They cover classic CNNs, modern CNNs, a window-attention transformer and a foundation model.

Table 5.1: Seven candidate architectures

Model	timm checkpoint	Reason for inclusion
EfficientNet-B0 [12]	efficientnet_b0.ra_in1k	Lightweight CNN; mid-term baseline
ResNet-50 [11]	resnet50.a1_in1k	Plain residual CNN reference
Xception [10]	legacy_xception.tf_in1k	Standard FaceForensics++ deepfake baseline
ConvNeXt-Tiny [14]	convnext_tiny.fb_in22k_ft_in1k	Modern CNN with ImageNet-22k pretraining
Swin-Tiny [21]	swin_tiny_patch4_window7_224.ms_in22k_ft_in1k	Window-attention transformer that keeps local detail
EfficientNet-B4 [12, 13]	tf_efficientnet_b4.ns_jft_in1k	Larger EfficientNet (Noisy Student), common in DFDC solutions
CLIP ViT-B/16 [22]	vit_base_patch16_clip_224.openai	Foundation model; only the last 4 of 12 blocks fine-tuned

5.3 Training Configuration

Every model used the same face images, split, augmentation, 224×224 input and epoch budget. The shared settings are listed in Table 5.2; the settings that differ by model family are listed in Table 5.3.

Table 5.2: Shared training settings

Setting	Value
Input size / batch size	224 × 224 / 64
Epoch budget / early-stopping patience	8 / 3
Optimiser	AdamW [37]
Learning rate of the head	1e-3
Weight decay on biases and normalisation parameters	0
Schedule	Linear warm-up for 5% of steps, then cosine decay [38]
Warm-up phase	Epoch 1: zero-initialised head only; later epochs: backbone too
Dropout before the head	0.3
Gradient clipping	Global norm 1.0
Precision	Automatic mixed precision (FP16) [39]
Loss	BCEWithLogitsLoss, pos_weight = 0.508
Head initialization	Zero weights and bias (starts from a neutral logit)
Checkpoint criterion	Best validation frame-level AUC
Random seed	42 (Python, NumPy, PyTorch)

Table 5.3: Model-specific settings and training outcome (Run A, Kaggle T4)

Model	Backbone LR	Weight decay	Drop path	Fine-tuned params	Train time (min)	Best epoch
EfficientNet-B0	2e-4	1e-4	0.1	4.0 M	21.3	7
ResNet-50	2e-4	1e-4	0.1	23.5 M	31.1	7
Xception	2e-4	1e-4	-	20.8 M	47.6	8
ConvNeXt-Tiny	1e-4	0.05	0.1	27.8 M	33.3	8
Swin-Tiny	1e-4	0.05	0.1	27.5 M	40.9	5
EfficientNet-B4	2e-4	1e-4	0.2	17.6 M	35.0	3*
CLIP ViT-B/16	2e-5	0.05	0.1	28.4 of 85.8 M	39.3	8

* EfficientNet-B4 stopped early after six epochs, because the validation AUC did not improve for three epochs after epoch 3. BatchNorm CNNs keep the EfficientNet-B0 recipe; ConvNeXt, Swin and CLIP use lower learning rates and a weight decay of 0.05.

5.4 Training Procedure

1. Build the pretrained backbone with a single-output head and set the head weights to zero.
2. Epoch 1 (warm-up): freeze the backbone and train only the head, so that the random head does not disturb the pretrained features.
3. Later epochs: unfreeze the backbone (for CLIP only the last four blocks); the head and backbone use different learning rates.

4. After each epoch, score the validation faces, compute frame-level and video-level validation AUC and EER, and save best.pt if the frame-level AUC improved.
5. Save the full training state (model, optimiser, scheduler, scaler) to last.pt after every epoch, so that an interrupted Kaggle session resumes where it stopped.
6. Stop early if the AUC does not improve for three epochs; write the epoch history to history.csv.
7. Reload best.pt, score the validation, holdout and test faces, choose the threshold and save the predictions and metrics; no retraining is done during evaluation.

5.5 Video-Level Aggregation and Threshold Selection

A video v with detected faces $F(v)$ receives the score

$$s(v) = (1 / |F(v)|) \cdot \sum p_i, \quad i \in F(v),$$

that is, the mean fake probability over its faces. The video is predicted FAKE if $s(v) \geq t$. The threshold t is chosen on the validation videos as the point on the ROC curve that maximises Youden’s $J = \text{TPR} - \text{FPR}$ [32], and is reused unchanged for the holdout and test splits. The holdout and test splits are never used for choosing weights or thresholds, so their results remain independent.

5.6 Evaluation Protocol of the Final Comparison

- **Confidence intervals:** a stratified bootstrap [31] with 1,000 resamples of the holdout real and fake videos gives a 95% interval for each holdout AUC; gaps smaller than the interval are treated as noise.
- **Speed:** GPU throughput is measured on batches of 64 images of size 224×224 in FP16 (3 warm-up batches, then 20 timed batches), so it excludes decoding and face detection.
- **Robustness:** up to 8 holdout faces per video are scored again after JPEG re-compression at quality 50 and 30, downscaling to half size and back, and Gaussian blur (radius 1.5); the worst AUC drop against the clean score is reported.
- **Calibration:** one temperature T per model is fitted on the validation frames by minimising log loss over 300 values between 0.1 and 100, and the holdout video log loss and expected calibration error (ECE, 15 bins) are compared before and after scaling [33].
- **Explainability:** Grad-CAM [29] is computed for one real and one fake holdout face for every model, from each model’s last spatial layer (for CLIP, the patch tokens of the last block reshaped to 14×14).
- **Demo and export:** the best model scores the first holdout video of each class end to end (32 frames), and the weights, threshold and pre-processing settings are saved as one inference bundle.

5.7 ConvNeXt-Tiny Rebuild (Run B, Run B)

To test more frames and low-resolution input, ConvNeXt-Tiny was rebuilt independently as Run B within the final Kaggle workflow. It keeps the same data pipeline and ConvNeXt-Tiny settings used for the comparison. The executed run trained for eight epochs (0–7); the best validation frame AUC occurred at epoch 5 and that checkpoint was selected for evaluation.

Table 5.4: Input options of the final Kaggle notebook

Option	Input	Enhancement test
1	Celeb-DF v2 attached in Kaggle, or downloaded with kagglehub	1 low-resolution FAKE video
2	Celeb-DF v2 attached through Kaggle ‘Add Input’	3 different low-resolution REAL videos
3	One personal video uploaded to Kaggle (needs an attached best.pt)	1 personal video

Table 5.5 and Figure 5.1 show the Run B training history. The warm-up epoch started near 0.59 validation frame AUC; after the backbone was unfrozen, validation performance rose rapidly above 0.98. The best validation frame AUC was 0.9976 at epoch 5.

Table 5.5: Run B training history (ConvNeXt-Tiny, Kaggle T4)

Epoch	Phase	Train loss	Val. frame AUC	Val. video AUC	Minutes
1	head-only	0.3890	0.5826	0.5976	3.37
2	full	0.1169	0.9895	0.9981	5.00
3	full	0.0372	0.9913	0.9986	4.55
4	full	0.0208	0.9938	0.9979	4.52
5	full	0.0138	0.9957	0.9993	4.44
6	full	0.0089	0.9973	0.9998	4.44
7	full	0.0046	0.9973	0.9997	4.41
8	full	0.0035	0.9971	0.9997	4.40



Figure 5.1: Run B training loss (a) and validation AUC (b); epoch 5 was selected

Total Run B training time was approximately 30.2 minutes (733 batches per epoch). The decision threshold chosen only on the validation videos with Youden's J was $t = 0.4825$.

5.8 Low-Resolution 64-Frame Enhancement Test (Run B)

This test asks whether enhancing low-resolution frames improves the result. It was designed so that the only difference between the two conditions is the enhancement step:

- **Selection:** official-test videos of the chosen class are read, sorted by pixel area (width $\times$ height), and the smallest ones are taken. With Option 2 these are the three lowest-resolution REAL videos.
- **Frames:** 64 evenly spaced frame positions per video, identical for both conditions.
- **Model and threshold:** the same independently trained Run B ConvNeXt-Tiny checkpoint and the same validation-selected threshold (0.4825) were used for Original and Enhanced conditions.
- **Enhancement:** applied to each frame before RetinaFace in the Enhanced condition only (Table 5.6).

- **Output:** frames read, faces found, face-detection rate, fake score, confidence, prediction and, when the label is known, whether the prediction is correct.

Table 5.6: Frame enhancement steps

Step	Operation	Details
1	Upscaling	If the frame is less than 360 pixels high, resize it with bicubic interpolation so that its height is 360 pixels
2	Contrast (CLAHE [40])	Convert to YCrCb and apply CLAHE (clip limit 1.5, 8×8 tiles) to the luminance channel only, so that colours are not changed
3	Sharpening	Unsharp mask: $1.15 \times \text{image} - 0.15 \times \text{Gaussian-blurred image}$ ($\sigma = 1.0$)

The confidence reported for a video is the probability of the predicted class: the fake score $s(v)$ when the prediction is FAKE and $1 - s(v)$ when it is REAL.

5.9 High- versus Low-Resolution Check (Run A)

The final notebook adds a short check of how the selected model behaves on videos of very different native resolution. It does not change any earlier training or results:

- **Selection:** all readable Celeb-DF videos are sorted by pixel area within each class; the smallest and the largest REAL and FAKE videos are taken, giving exactly four videos (manual overrides are possible).
- **Model and threshold:** the best Run A model (ConvNeXt-Tiny) with its validation-selected threshold (0.0079).
- **Frames:** 32 evenly spaced frames per video, processed with the same RetinaFace detection and alignment; no enhancement is applied.
- **Output:** faces detected, mean $P(\text{fake})$, prediction, correctness and time, together with six of the aligned crops for each video.

Chapter 6: Code and Screenshots

This chapter shows the key parts of the two notebooks used in the project, together with the outputs they produced: the team's final notebook (deepfake-detection-final.ipynb, Run A) and the low-resolution notebook (deepfake-detection-low-resolution-64-frame-final.ipynb, Run B). Code is abridged for space. Maps of all notebook cells are given in Appendix B.

6.1 Final Team Notebook (Run A)

6.1.1 Environment, Data and Model Registry

Output 6.1: Environment, dataset, split and face-cache checks (Sections 1-4)

```
torch 2.11.0+cu128 | timm 1.0.11 | onnxruntime 1.26.0 | Tesla T4
Dataset: /kaggle/input/datasets/reubensuju/celeb-df-v2
Dataset ready: {'Celeb-real': 590, 'YouTube-real': 300, 'Celeb-synthesis': 5639}
train / val / holdout are identity-disjoint. The official test shares 36 identities with
train (expected for Celeb-DF).

```

	real	fake	youtube_real	fake_per_real
train	495	3890	171	7.8586
val	112	705	35	6.2946
holdout	105	704	24	6.7048
test	178	340	70	1.9101

```
73,429 face crops | 0 videos without a detected face
pos_weight = 0.508
```

Listing 6.1: Model registry of the seven architectures (Section 1)

```
@dataclass(frozen=True)
class ModelSpec:
    timm_name: str
    lr_backbone: float
    weight_decay: float
    cam_layer: str
    drop_path_rate: float | None = 0.1
    trainable_blocks: int | None = None
    model_kwargs: dict = field(default_factory=dict)

MODELS = {
    "efficientnet_b0": ModelSpec("efficientnet_b0.ra_in1k", 2e-4, 1e-4, "bn2"),
    "resnet50": ModelSpec("resnet50.a1_in1k", 2e-4, 1e-4, "layer4"),
    "xception": ModelSpec("legacy_xception.tf_in1k", 2e-4, 1e-4, "act4", drop_path_rate=None),
    "convnext_tiny": ModelSpec("convnext_tiny.fb_in22k_ft_in1k", 1e-4, 0.05, "stages.3"),
    "swin_tiny": ModelSpec("swin_tiny_patch4_window7_224.ms_in22k_ft_in1k", 1e-4, 0.05,
                           "layers.3"),
    "efficientnet_b4": ModelSpec("tf_efficientnet_b4.ns_jft_in1k", 2e-4, 1e-4, "bn2",
                                drop_path_rate=0.2),
    "clip_vit_b16": ModelSpec("vit_base_patch16_clip_224.openai", 2e-5, 0.05, "blocks.11.norm1",
                              trainable_blocks=4, model_kwargs={"act_layer": "quick_gelu"}),
}
```

6.1.2 Training and Evaluation of the Seven Models

Listing 6.2: Training epoch and evaluation of one run (Sections 4-5, abridged)

```
def _run_epoch(self) -> None:
    start, warmup = time.time(), self.state.epoch < CFG.freeze_epochs
    set_phase(self.model, self.spec, warmup) # head only in epoch 0
    train_loss = train_one_epoch(self.model, self.train_loader, self.optimizer,
                                self.scheduler, self.scaler, self.criterion)
    val = score_predictions(SPLIT_FRAMES["val"].assign(
        prob=predict(self.model, self.val_loader)), threshold=0.5)
    improved = val["frame_auc"] > self.state.best_auc
    if improved: # keep the best epoch
        self.state.best_auc = val["frame_auc"]
        atomic_save(self.model.state_dict(), self.out / "best.pt")
    self.state.bad_epochs = 0 if improved else self.state.bad_epochs + 1

def evaluate_run(key, seed):
    model, eval_tf = load_best(key, seed)
    predictions = {split: SPLIT_FRAMES[split].assign(
        prob=predict(model, make_loader(SPLIT_FRAMES[split], eval_tf)))
        for split in ("val", "holdout", "test")}
    threshold = youden_threshold(predictions["val"].label, predictions["val"].prob)
    ... # video-level metrics per split, GPU throughput, best epoch -> metrics.json
```

Output 6.2: ConvNeXt-Tiny training log and final AUC of every model (Section 5, abridged)

```
convnext_tiny: 27.82M params | 733 steps/epoch
epoch 0 warmup | loss 0.3909 | val frame AUC 0.5867 (video 0.5990, EER 0.4233) | 2.9 min
epoch 1 full | loss 0.1171 | val frame AUC 0.9873 (video 0.9967, EER 0.0370) | 4.3 min
epoch 2 full | loss 0.0355 | val frame AUC 0.9944 (video 0.9986, EER 0.0167) | 4.0 min
epoch 3 full | loss 0.0219 | val frame AUC 0.9946 (video 0.9995, EER 0.0174) | 3.9 min
epoch 4 full | loss 0.0128 | val frame AUC 0.9963 (video 0.9997, EER 0.0073) | 3.9 min
epoch 5 full | loss 0.0083 | val frame AUC 0.9976 (video 0.9998, EER 0.0073) | 3.9 min
epoch 6 full | loss 0.0051 | val frame AUC 0.9973 (video 0.9996, EER 0.0059) | 3.9 min
epoch 7 full | loss 0.0043 | val frame AUC 0.9974 (video 0.9997, EER 0.0094) | 3.9 min
Best checkpoint: epoch 5.
```

deepfake-video-detection-final-capstone-run-a-r... Draft saved

File Edit View Run Settings Add-ons Help

+ ▾

 Run All

Code ▾

```
model.safetensors: reconstructing file: 100% ██████████ 114MB / 114MB, 11.1MB/s
model.safetensors: downloading bytes: ██████████ 108MB, 10.4MB/s
convnext_tiny: 27.8M params (27.8M fine-tuned), 733 steps/epoch
Error displaying widget: model not found
Error displaying widget: model not found
epoch 0 warmup | loss 0.3909 | val AUC 0.5867 (video 0.5990, EER 0.4233) | 2.9 min *
Error displaying widget: model not found
Error displaying widget: model not found
epoch 1 full | loss 0.1171 | val AUC 0.9873 (video 0.9967, EER 0.0370) | 4.3 min *
Error displaying widget: model not found
Error displaying widget: model not found
epoch 2 full | loss 0.0355 | val AUC 0.9944 (video 0.9986, EER 0.0167) | 4.0 min *
Error displaying widget: model not found
Error displaying widget: model not found
epoch 3 full | loss 0.0219 | val AUC 0.9946 (video 0.9995, EER 0.0174) | 3.9 min *
Error displaying widget: model not found
Error displaying widget: model not found
epoch 4 full | loss 0.0128 | val AUC 0.9963 (video 0.9997, EER 0.0073) | 3.9 min *
Error displaying widget: model not found
Error displaying widget: model not found
epoch 5 full | loss 0.0083 | val AUC 0.9976 (video 0.9998, EER 0.0073) | 3.9 min *
Error displaying widget: model not found
Error displaying widget: model not found
epoch 6 full | loss 0.0051 | val AUC 0.9973 (video 0.9996, EER 0.0059) | 3.9 min
Error displaying widget: model not found
Error displaying widget: model not found
epoch 7 full | loss 0.0043 | val AUC 0.9974 (video 0.9997, EER 0.0094) | 3.9 min
Error displaying widget: model not found
Error displaying widget: model not found
Error displaying widget: model not found
convnext_tiny: holdout video AUC 0.9998 | test video AUC 0.9990

model.safetensors: reconstructing file: 100% ██████████ 114MB / 114MB, 10.9MB/s
model.safetensors: downloading bytes: ██████████ 107MB, 10.0MB/s
swin_tiny: 27.5M params (27.5M fine-tuned), 733 steps/epoch
Error displaying widget: model not found
Error displaying widget: model not found
epoch 0 warmup | loss 0.3735 | val AUC 0.5757 (video 0.5810, EER 0.4546) | 2.8 min *
Error displaying widget: model not found
Error displaying widget: model not found
epoch 1 full | loss 0.1054 | val AUC 0.9857 (video 0.9927, EER 0.0384) | 5.0 min *
Error displaying widget: model not found
Error displaying widget: model not found
epoch 2 full | loss 0.0397 | val AUC 0.9920 (video 0.9978, EER 0.0189) | 5.0 min *
```

6.1.3 Comparison, Robustness and Calibration

Listing 6.3: Bootstrap interval, robustness conditions and temperature scaling (Sections 6-8)

```
def bootstrap_auc_ci(videos, n_boot=1000, seed=0):
    rng = np.random.default_rng(seed)
    real = videos.prob[videos.label == 0].to_numpy()
    fake = videos.prob[videos.label == 1].to_numpy()
    targets = np.r_[np.zeros(len(real)), np.ones(len(fake))]
    aucs = [roc_auc_score(targets, np.r_[rng.choice(real, len(real)),
                                         rng.choice(fake, len(fake))]) for _ in
            range(n_boot)]
    return tuple(np.percentile(aucs, [2.5, 97.5]))          # stratified 95% interval

PERTURBATIONS = {"clean": identity,
                  "jpeg_q50": functools.partial(jpeg_compress, quality=50),
                  "jpeg_q30": functools.partial(jpeg_compress, quality=30),
                  "downscale_x0.5": downscale, "blur_r1.5": blur}

def fit_temperature(labels, probs):                        # one T per model
    logits, grid = to_logits(probs), np.logspace(-1, 2, 300)
    losses = [log_loss(labels, expit(logits / t), labels=[0, 1]) for t in grid]
    return float(grid[int(np.argmin(losses))])
```

Output 6.3: Model comparison and robustness tables (Sections 6-7, abridged columns)

model	Params(M)	GPU img/s	Train min	Holdout frame AUC	Holdout video AUC	
CI low	CI high	EER	Holdout F1	Test AUC	Test F1	
convnext_tiny	27.8209	643.2474	30.8771	0.9963	0.9998	0.9993
1.0000	0.0076	0.9902	0.9990	0.9497		
efficientnet_b4	17.5504	521.3066	31.2077	0.9919	0.9994	0.9986
0.9999	0.0119	0.9922	0.9992	0.9869		
xception	20.8090	568.9492	29.8581	0.9902	0.9991	0.9981
0.0188	0.9943	0.9991	0.9700			
swin_tiny	27.5201	457.4665	37.2300	0.9928	0.9984	0.9964
0.0188	0.9901	0.9989	0.9471			0.9997
efficientnet_b0	4.0088	1479.6056	22.1450	0.9891	0.9981	0.9963
0.9994	0.0133	0.9880	0.9978	0.9524		
resnet50	23.5101	780.3894	27.7604	0.9896	0.9976	0.9956
0.0188	0.9901	0.9981	0.9671			0.9991
clip_vit_b16	85.8002	350.3646	33.7194	0.9777	0.9905	0.9839
0.9959	0.0389	0.9772	0.9959	0.9354		

deepfake-video-detection-final-capstone-run-a-r... Draft saved

File Edit View Run Settings Add-ons Help

+ - ✂ 📄 ▶ ⏮ Run All Code -

```

"holdout_ci_low": "CI low",
"holdout_ci_high": "CI high",
"holdout_video_eer": "Holdout video EER",
"holdout_video_f1": "Holdout video F1",
"test_video_auc": "Test video AUC",
"test_video_f1": "Test video F1",
}
SUMMARY = (
    RESULTS.groupby("model", sort=False)[list(SUMMARY_COLUMNS)].mean()
    .assign(seeds=RESULTS.groupby("model", sort=False).size())
    .rename(columns=SUMMARY_COLUMNS)
    .sort_values(["Holdout video AUC", "Holdout video EER"], ascending=[False, True])
)
SUMMARY

```

	Params (M)	GPU img/s	Train min	Holdout frame AUC	Holdout video AUC	CI low	CI high	Holdout video EER	Holdout video F1	Test video AUC	Test video F1	seeds
convnext_tiny	27.8209	643.2474	30.8771	0.9963	0.9998	0.9993	1.0000	0.0076	0.9902	0.9990	0.9497	1
efficientnet_b4	17.5504	521.3066	31.2077	0.9919	0.9994	0.9986	0.9999	0.0119	0.9922	0.9992	0.9869	1
xception	20.8090	568.9492	29.8581	0.9902	0.9991	0.9981	0.9998	0.0188	0.9943	0.9991	0.9700	1
swin_tiny	27.5201	457.4665	37.2300	0.9928	0.9984	0.9964	0.9997	0.0188	0.9901	0.9989	0.9471	1
efficientnet_b0	4.0088	1479.6056	22.1450	0.9891	0.9981	0.9963	0.9994	0.0133	0.9880	0.9978	0.9524	1
resnet50	23.5101	780.3894	27.7604	0.9896	0.9976	0.9956	0.9991	0.0188	0.9901	0.9981	0.9671	1
clip_vit_b16	85.8002	350.3466	33.7194	0.9777	0.9905	0.9839	0.9959	0.0389	0.9772	0.9959	0.9354	1

+ Code + Markdown

```

[39]: if RESULTS.seed.nunique() > 1:
      display(RESULTS.groupby("model", sort=False)[["holdout_video_auc", "holdout_video_eer", "test_video_auc"]].agg(["mean", "std", "count"]))

```

6.1.4 Demo, Export and High / Low Resolution Check

Output 6.4: End-to-end demo, export and resolution check (Sections 10-11)

Best model: convnext_tiny | threshold 0.0079

video	truth	faces	p_fake	prediction	seconds
id49_0000.mp4	REAL	32	3.6828e-05	REAL	6.0812
id49_id50_0000.mp4	FAKE	32	7.4542e-01	FAKE	10.5001

Participant 4 — High vs Low Resolution Comparison

REAL_HIGH	id16_0012.mp4	974x500	32 faces	P(fake)=0.0000	REAL	correct
FAKE_HIGH	id16_id9_0012.mp4	974x500	32 faces	P(fake)=1.0000	FAKE	correct
REAL_LOW	id24_0003.mp4	286x300	32 faces	P(fake)=0.0019	REAL	correct
FAKE_LOW	id24_id19_0003.mp4	286x300	32 faces	P(fake)=0.9999	FAKE	correct

Accuracy: 100.0% (4/4)

```
[49]: def predict_video(path: Path, model: nn.Module, eval_tf, threshold: float, count: int = 32) -> dict:
      crops = extract_faces(path, count)
      if not crops:
          return {"faces": 0, "p_fake": float("nan"), "prediction": "NO FACE"}
      batch = torch.stack([eval_tf(Image.fromarray(cv2.cvtColor(crop, cv2.COLOR_BGR2RGB)))] for crop in crops])
      with torch.no_grad(), torch.autocast("cuda", dtype=torch.float16):
          p_fake = torch.sigmoid(model(batch.to(DEVICE))).float().mean().item()
      return {"faces": len(crops), "p_fake": p_fake, "prediction": "FAKE" if p_fake >= threshold else "REAL"}

      BEST_MODEL = SUMMARY.index[0]
      best_model, best_tf = load_best(BEST_MODEL, SEED)
      best_threshold = float(PRIMARY.set_index("model").loc[BEST_MODEL, "threshold"])
      video_labels = VIDEOS.set_index("stem").label

      demo_rows = []
      for path in sorted(CFG.demo_dir.glob("*.mp4")):
          start = time.time()
          result = predict_video(path, best_model, best_tf, best_threshold)
          demo_rows.append({"video": path.name, "truth": CLASS_NAMES[video_labels[path.stem]].upper(),
                           **result, "seconds": time.time() - start})
      print(f"Best model: {BEST_MODEL} | threshold {best_threshold:.4f}")
      pd.DataFrame(demo_rows)
```

Best model: convnext_tiny | threshold 0.0079

```
[49]:
```

	video	truth	faces	p_fake	prediction	seconds
0	id49_0000.mp4	REAL	32	3.6828e-05	REAL	6.0812
1	id49_id50_0000.mp4	FAKE	32	7.4542e-01	FAKE	10.5001

```
[55]: # Compact comparison for report / viva
P4_SUMMARY = P4_RESULTS[["case", "truth", "resolution", "faces_detected", "p_fake", "prediction", "correct"]].copy()
P4_SUMMARY["p_fake"] = P4_SUMMARY["p_fake"].round(4)
print("Participant 4 - High vs Low Resolution Comparison")
display(P4_SUMMARY)

valid = P4_RESULTS[P4_RESULTS.prediction.ne("NO FACE")].copy()
if len(valid):
    print(f"Accuracy on these 4 demonstration videos: {valid.correct.mean():.1%} ({int(valid.correct.sum())}/{len(valid)})")
else:
    print("No videos produced a usable prediction; inspect face detection and input videos.")

# Save only this participant's table; earlier participants' artifacts/outputs are untouched.
try:
    p4_out_dir = CFG.drive_root
    p4_out_dir.mkdir(parents=True, exist_ok=True)
    p4_csv = p4_out_dir / "participant4_resolution_4video_results.csv"
    P4_RESULTS.to_csv(p4_csv, index=False)
    print(f"Saved: {p4_csv}")
except Exception as exc:
    print(f"Result table displayed successfully; CSV save skipped: {exc}")
```

Participant 4 - High vs Low Resolution Comparison

	case	truth	resolution	faces_detected	p_fake	prediction	correct
0	REAL_HIGH	REAL	974x500	32	0.0000	REAL	True
1	FAKE_HIGH	FAKE	974x500	32	1.0000	FAKE	True
2	REAL_LOW	REAL	286x300	32	0.0019	REAL	True
3	FAKE_LOW	FAKE	286x300	32	0.9999	FAKE	True

Accuracy on these 4 demonstration videos: 100.0% (4/4)

Saved: /kaggle/working/Deepfake_Capstone/retinaface_models/participant4_resolution_4video_results.csv

[+ Code](#) [+ Markdown](#)

Participant 4 interpretation

Use the table above to compare whether the selected model remains correct when the native video resolution is lower. The displayed aligned face crops are the actual face inputs extracted by discussed as evidence that resolution affects the end-to-end pipeline.

Important: this is a four-video demonstration, not a statistically representative accuracy benchmark. The broader robustness experiment in Section 7 remains the stronger aggregate test.

6.2 Low-Resolution Run B (Run B)

6.2.1 Environment and GPU Check

Listing 6.4: PyTorch, GPU and ONNX Runtime pre-flight check (Cells 0 and 2)

```
import torch
import onnxruntime as ort
import timm

assert torch.cuda.is_available(), "Switch Kaggle runtime to a GPU accelerator."
DEVICE = torch.device("cuda")
ORT_PROVIDERS = ort.get_available_providers()
print(f'torch {torch.__version__} | timm {timm.__version__} | onnxruntime
{ort.__version__} | {torch.cuda.get_device_name(0)}')
print("ONNX Runtime providers:", ORT_PROVIDERS)
print("Environment preflight: PASS")
```

Output 6.5

```
torch 2.11.0+cu128 | timm 1.0.29 | onnxruntime 1.30.0 | Tesla T4
ONNX Runtime providers: ['AzureExecutionProvider', 'CPUExecutionProvider']
Environment preflight: PASS
```

PyTorch training used the Tesla T4. RetinaFace used ONNX Runtime CPUExecutionProvider in this final Kaggle execution.

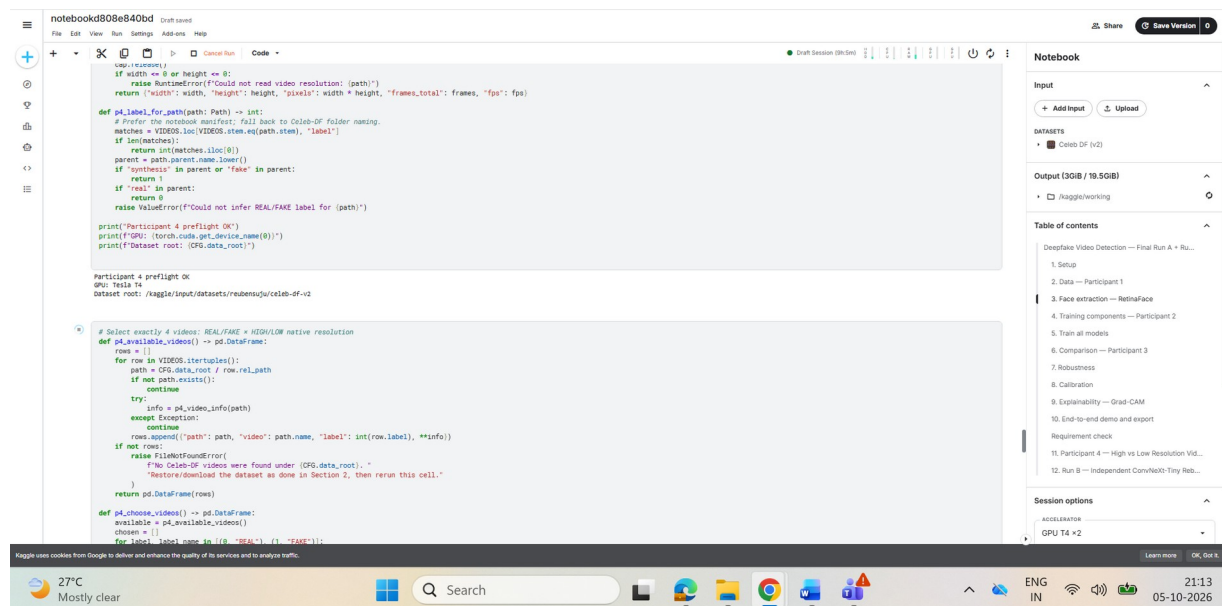


Figure 6.1: Kaggle screenshot of the final Kaggle notebook: environment preflight showing Tesla T4 and ONNX Runtime providers (5 October 2026)

6.2.2 Manifest and Leakage Audit

Listing 6.5: Identity components (union-find) and leakage audit (Cell 5)

```
def identity_components(videos):
    parent = {}
    def find(i):
        parent.setdefault(i, i)
        while parent[i] != i:
            parent[i] = parent[parent[i]]; i = parent[i]
        return i
    def union(a, b):
        ra, rb = find(a), find(b)
        if ra != rb: parent[rb] = ra
    for id_a, id_b in zip(videos.id_a, videos.id_b):
        find(id_a)
        if isinstance(id_b, str): union(id_a, id_b)
    return videos.id_a.map(find)

def audit_leakage(videos):
    for a, b in [("train", "val"), ("train", "holdout"), ("val", "holdout")]:
        shared = split_identities(videos, a) & split_identities(videos, b)
        if shared:
            raise RuntimeError(f"Identity leakage between {a} and {b}")
    print("Leakage audit passed: train / val / holdout are identity-disjoint.")
```

Output 6.6

```
Celeb-real: 590 MP4 videos
YouTube-real: 300 MP4 videos
Celeb-synthesis: 5639 MP4 videos
Leakage audit passed: train / val / holdout are identity-disjoint.

      real  fake
split
train    495 3890
val      112 705
holdout  105 704
test     178 340
```

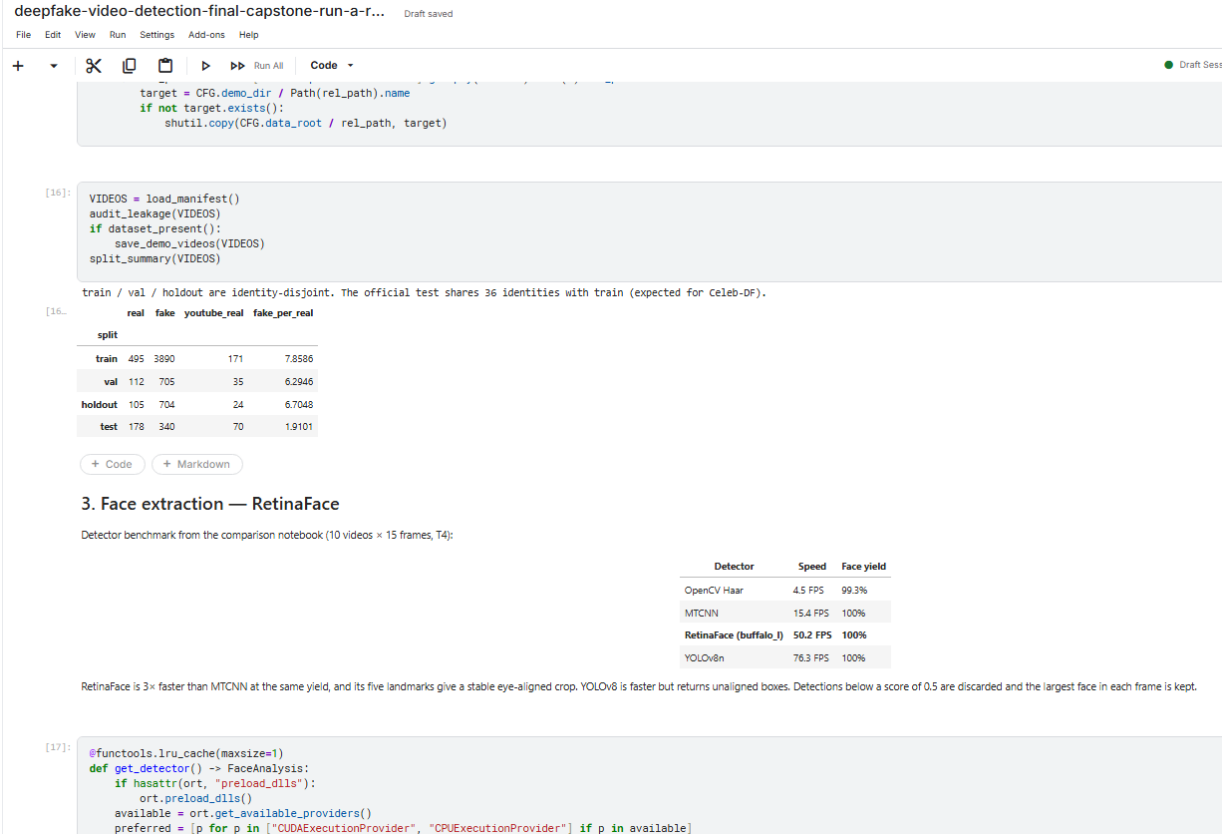


Figure 6.2: Kaggle screenshot of the final Kaggle notebook: manifest and leakage audit

6.2.3 RetinaFace Detection and Eye Alignment

Listing 6.6: Eye alignment and face extraction (Cell 6)

```
def align_face(image, landmarks, size):
    # size = 256
    left_eye, right_eye = landmarks[0], landmarks[1]
    dx, dy = (right_eye - left_eye).tolist()
    distance = math.hypot(dx, dy)
    if distance < 1e-3:
        return None
    center = tuple(((left_eye + right_eye) / 2).tolist())
    scale = (1 - 2 * EYE_MARGIN_X) * size / distance # EYE_MARGIN_X = 0.35
    M = cv2.getRotationMatrix2D(center, math.degrees(math.atan2(dy, dx)), scale)
    M[0, 2] += size * 0.5 - center[0]
    M[1, 2] += size * EYE_LINE_Y - center[1] # EYE_LINE_Y = 0.35
    return cv2.warpAffine(image, M, (size, size), flags=cv2.INTER_CUBIC,
                          borderMode=cv2.BORDER_REPLICATE)

def extract_face_from_frame(frame):
    face = largest_face(get_detector().get(frame)) # RetinaFace, det_thresh 0.5
    return None if face is None else align_face(frame, face.kps, CFG.align_size)
```

Output 6.7

RetinaFace providers: ['CPUExecutionProvider']

Face-cache shard 1/12: 545/545 [24:57]

Face-cache shard 2/12: 544/544 [25:11]

Face-cache shard 3/12: 544/544 [23:00]

Face-cache shard 4/12: 544/544 [24:15]

Face-cache shard 5/12: 544/544 [42:37]

Face-cache shard 6/12: 544/544 [24:10]

Face-cache shard 7/12: 544/544 [22:38]

Face-cache shard 8/12: 544/544 [23:04]

Face-cache shard 9/12: 544/544 [24:30]

Face-cache shard 10/12: 544/544 [24:19]

Face-cache shard 11/12: 544/544 [24:03]

Face-cache shard 12/12: 544/544 [23:55]

73,429 aligned face crops indexed.

label	real	fake
train	15805	31120
val	3584	5640
holdout	3360	5632
test	2848	5440

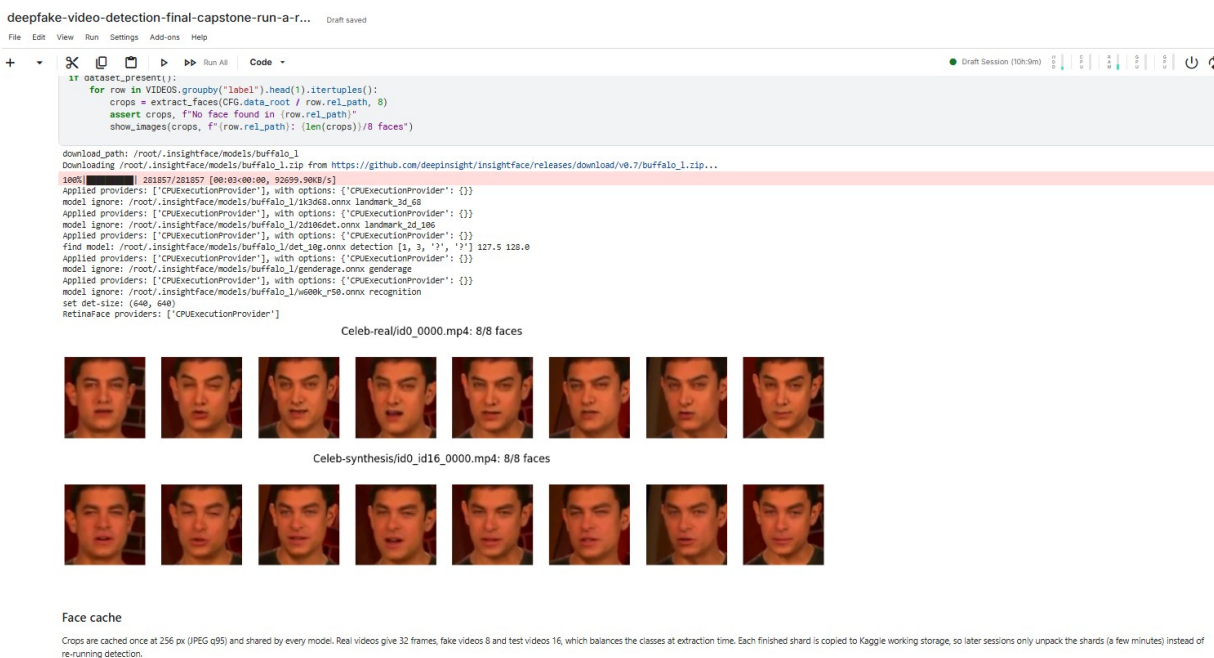


Figure 6.3: Kaggle screenshot of the final Kaggle notebook: RetinaFace model download and the first face-cache shard

```
def ensure_face_cache(videos: pd.DataFrame) -> None:
    for split in SPLITS:
        for name in CLASS_NAMES:
            (CFG.cache_dir / split / name).mkdir(parents=True, exist_ok=True)
        restore_shards()
        for index in range(CFG.cache_shards):
            if not CFG.shard_path(index).exists():
                build_shard(videos, index)
```

[23]: ensure_face_cache(VIDEOS)

Shard 1/12: 100%		545/545	[24:57<00:00, 2.97s/it]
Shard 2/12: 100%		544/544	[25:11<00:00, 2.63s/it]
Shard 3/12: 100%		544/544	[23:00<00:00, 1.84s/it]
Shard 4/12: 100%		544/544	[24:15<00:00, 2.17s/it]
Shard 5/12: 100%		544/544	[24:37<00:00, 5.46s/it]
Shard 6/12: 100%		544/544	[24:10<00:00, 2.60s/it]
Shard 7/12: 100%		544/544	[22:38<00:00, 2.75s/it]
Shard 8/12: 100%		544/544	[23:04<00:00, 1.90s/it]
Shard 9/12: 100%		544/544	[24:30<00:00, 3.05s/it]
Shard 10/12: 100%		544/544	[24:19<00:00, 3.55s/it]
Shard 11/12: 100%		544/544	[24:03<00:00, 2.24s/it]
Shard 12/12: 100%		544/544	[23:55<00:00, 2.26s/it]

[24]:

```
def index_face_cache(videos: pd.DataFrame) -> pd.DataFrame:
    paths = sorted(CFG.cache_dir.glob("*/*/*.jpg"))
    frames = pd.DataFrame({
        "path": [str(p) for p in paths],
        "stem": [p.stem.rsplit("_f", 1)[0] for p in paths],
        "folder_split": [p.parent.parent.name for p in paths],
    })
    frames = frames.merge(videos[["stem", "label", "split"]], on="stem", how="left", validate="many_to_one")
    assert frames.label.notna().all(), "The cache contains crops without a source video"
    assert (frames.folder_split == frames.split).all(), "Cache folders disagree with the manifest"
    return frames.drop(columns="folder_split").astype({"label": int})
```

Figure 6.4: Kaggle screenshot of the final Kaggle notebook: remaining face-cache shards and crop counts

6.2.4 Model Construction and Training Loop

Listing 6.7: ConvNeXt-Tiny construction and training loop (Cells 8 and 10, abridged)

```
def build_model(pretrained=True):
    model = timm.create_model("convnext_tiny.fb_in22k_ft_in1k", pretrained=pretrained,
                              num_classes=1, drop_rate=0.3, drop_path_rate=0.1)
    if pretrained:
        head = model.get_classifier()
        nn.init.zeros_(head.weight); nn.init.zeros_(head.bias)
    return model.to(DEVICE)

criterion = nn.BCEWithLogitsLoss(pos_weight=torch.tensor([POS_WEIGHT], device=DEVICE))
for epoch in range(CFG.epochs):
    set_phase(model, warmup=epoch < CFG.freeze_epochs)      # head-only in epoch 1
    train_loss = train_one_epoch(model, train_loader, optimizer, scheduler,
                                  scaler, criterion)          # AMP + grad clipping
    val_frames["prob"] = predict_loader(model, val_loader)
    val_videos = video_scores(val_frames)                    # mean P(fake) per video
    val_frame_auc = roc_auc_score(val_frames.label, val_frames.prob)
    if val_frame_auc > best_val_auc:                          # keep best checkpoint
        best_val_auc, bad_epochs = val_frame_auc, 0
        torch.save(model.state_dict(), BEST_MODEL_PATH)
    else:
        bad_epochs += 1
    if bad_epochs >= CFG.patience:
        break
```

Output 6.8 (values rounded to four decimals)

```
Run B ConvNeXt-Tiny: 27.82M params | 733 steps/epoch
epoch 0 warmup | loss 0.3909 | val frame AUC 0.5867 | video 0.5990 | EER 0.4233 | 2.6 min
epoch 1 full   | loss 0.1171 | val frame AUC 0.9873 | video 0.9967 | EER 0.0370 | 4.0 min
epoch 2 full   | loss 0.0355 | val frame AUC 0.9944 | video 0.9986 | EER 0.0167 | 4.0 min
epoch 3 full   | loss 0.0219 | val frame AUC 0.9946 | video 0.9995 | EER 0.0174 | 4.0 min
epoch 4 full   | loss 0.0128 | val frame AUC 0.9963 | video 0.9997 | EER 0.0073 | 3.9 min
epoch 5 full   | loss 0.0083 | val frame AUC 0.9976 | video 0.9998 | EER 0.0073 | 3.9 min
epoch 6 full   | loss 0.0051 | val frame AUC 0.9973 | video 0.9996 | EER 0.0059 | 3.9 min
epoch 7 full   | loss 0.0043 | val frame AUC 0.9974 | video 0.9997 | EER 0.0094 | 3.9 min
Selected Run B epoch: 5 (best validation frame AUC = 0.9976)
```

```

        "model": model.state_dict(), "optimizer": optimizer.state_dict(),
        "scheduler": scheduler.state_dict(), "scaler": scaler.state_dict(),
        "epoch": epoch, "best_auc": best_auc, "bad_epochs": bad_epochs,
        "history": history,
    }, RUN_B_LAST)

    if not RUN_B_BEST.exists():
        raise RuntimeError("Run B training ended without creating best.pt")
    del model, train_loader, val_loader
    free_gpu()

if RUN_B_METRICS.exists() and RUN_B_BEST.exists():
    print("Run B training/evaluation already completed; existing independent checkpoint will be reused.")
else:
    run_b_train_or_resume()

```

Run B ConvNext-Tiny: 27.82M params | 733 steps/epoch

epoch 0	warmup	loss 0.3909	val AUC 0.5867 (video 0.5990, EER 0.4233)	2.6 min	*
epoch 1	full	loss 0.1171	val AUC 0.9873 (video 0.9967, EER 0.0370)	4.0 min	*
epoch 2	full	loss 0.0355	val AUC 0.9944 (video 0.9986, EER 0.0167)	4.0 min	*
epoch 3	full	loss 0.0219	val AUC 0.9946 (video 0.9995, EER 0.0174)	4.0 min	*
epoch 4	full	loss 0.0128	val AUC 0.9963 (video 0.9997, EER 0.0073)	3.9 min	*
epoch 5	full	loss 0.0083	val AUC 0.9976 (video 0.9998, EER 0.0073)	3.9 min	*
epoch 6	full	loss 0.0051	val AUC 0.9973 (video 0.9996, EER 0.0059)	3.9 min	
epoch 7	full	loss 0.0043	val AUC 0.9974 (video 0.9997, EER 0.0094)	3.9 min	

```

[58]: # RUN B - load the independent best checkpoint and compute threshold + validation/holdout metrics
RUN_B_MODEL = build_model(RUN_B_SPEC, pretrained=False)
RUN_B_MODEL.load_state_dict(torch.load(RUN_B_BEST, map_location=DEVICE, weights_only=True))
RUN_B_MODEL = RUN_B_MODEL.eval()
_, RUN_B_TF = build_transforms(RUN_B_MODEL)

RUN_B_PREDS = {}
for split in ("val", "holdout"):
    loader = make_loader(SPLIT_FRAMES[split], RUN_B_TF)
    probs = predict(RUN_B_MODEL, loader)
    RUN_B_PREDS[split] = SPLIT_FRAMES[split].assign(prob=probs)

# Threshold is selected ONLY on validation videos; holdout is never used for threshold tuning.
_run_b_val_videos = video_scores(RUN_B_PREDS["val"])
RUN_B_THRESHOLD = youden_threshold(_run_b_val_videos.label, _run_b_val_videos.prob)

RUN_B_EVAL = {}
for split in ("val", "holdout"):
    vids = video_scores(RUN_B_PREDS[split])
    RUN_B_EVAL[split] = classification_metrics(vids.label, vids.prob, RUN_B_THRESHOLD)

```

6.2.5 Threshold Selection and Evaluation

Listing 6.8: Validation threshold (Youden J) and video-level metrics (Cells 9 and 11)

```
def youden_threshold(y_true, y_prob):
    fpr, tpr, thresholds = roc_curve(y_true, y_prob)
    finite = np.isfinite(thresholds)
    fpr, tpr, thresholds = fpr[finite], tpr[finite], thresholds[finite]
    return float(thresholds[np.argmax(tpr - fpr)])

FINAL_THRESHOLD = youden_threshold(val_videos.label, val_videos.prob)
val_metrics = classification_metrics(val_videos.label, val_videos.prob, FINAL_THRESHOLD)
holdout_metrics = classification_metrics(holdout_videos.label, holdout_videos.prob,
                                        FINAL_THRESHOLD)
```

Output 6.9

Run B threshold (validation Youden J): 0.4825

Split	AUC	Accuracy	Precision	Recall	Specificity	F1	EER
Validation	0.9998	0.9878	1.0000	0.9858	1.0000	0.9929	0.0073
Holdout	0.9998	0.9728	1.0000	0.9688	1.0000	0.9841	0.0076

Confusion counts (TN / FP / FN / TP)

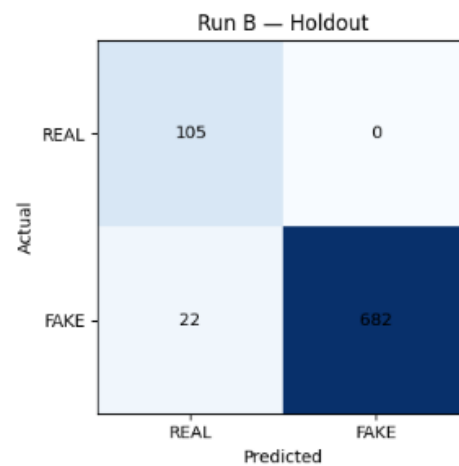
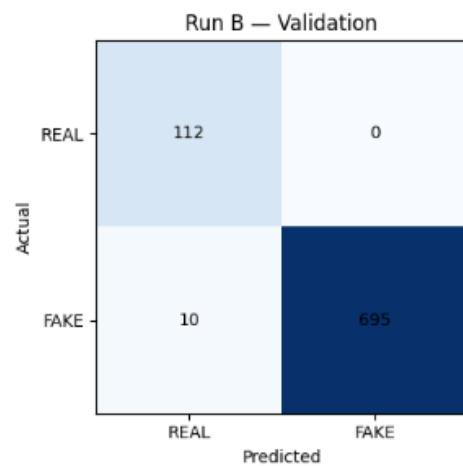
Validation: 112 / 0 / 10 / 695

Holdout: 105 / 0 / 22 / 682

```
pred = (vids.prob.to_numpy() >= RUN_B_THRESHOLD).astype(int)
cm = confusion_matrix(vids.label, pred, labels=[0, 1])
ax.imshow(cm, cmap="Blues")
ax.set_title(f"Run B - {title}")
ax.set_xlabel("Predicted")
ax.set_ylabel("Actual")
ax.set_xticks([0, 1], ["REAL", "FAKE"])
ax.set_yticks([0, 1], ["REAL", "FAKE"])
for (r, c), value in np.ndenumerate(cm):
    ax.text(c, r, str(value), ha="center", va="center")
plt.tight_layout()
plt.show()
```

Run B threshold (validation Youden J): 0.4825

	auc	ap	eer	accuracy	precision	recall	specificity	f1	tn	fp	fn	tp
split												
Validation	0.9998	1.0	0.0073	0.9878	1.0	0.9858	1.0	0.9929	112	0	10	695
Holdout	0.9998	1.0	0.0076	0.9728	1.0	0.9688	1.0	0.9841	105	0	22	682



+ Code + Markdown

```
[59]: # RUN B - training history plot (loss and validation AUC)
if not RUN_B_HISTORY.exists():
    raise FileNotFoundError(f"Missing Run B training history: {RUN_B_HISTORY}")
_run_b_hist = pd.read_csv(RUN_B_HISTORY)
if _run_b_hist.empty:
    raise RuntimeError("Run B history.csv is empty")
```

6.2.6 Enhancement and 64-Frame Video Inference

Listing 6.9: Frame enhancement and per-video inference (Cells 12 and 15, abridged)

```
def enhance_low_res_frame(frame_bgr, target_min_height=360):
    h, w = frame_bgr.shape[:2]
    if h < target_min_height:
        s = target_min_height / h
        frame_bgr = cv2.resize(frame_bgr, (round(w * s), round(h * s)),
                                interpolation=cv2.INTER_CUBIC)
    y, cr, cb = cv2.split(cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2YCrCb))
    y = cv2.createCLAHE(clipLimit=1.5, tileGridSize=(8, 8)).apply(y) # luminance only
    out = cv2.cvtColor(cv2.merge([y, cr, cb]), cv2.COLOR_YCrCb2BGR)
    blurred = cv2.GaussianBlur(out, (0, 0), 1.0)
    return cv2.addWeighted(out, 1.15, blurred, -0.15, 0) # mild sharpening

def predict_video(path, known_label, n_frames=64, enhance=False):
    crops, _, frames_read = extract_faces_from_video(path, n_frames, enhance)
    probs = score_faces(crops) # BGR -> RGB -> eval transform -> sigmoid
    fake_score = float(np.mean(probs))
    predicted = int(fake_score >= FINAL_THRESHOLD)
    confidence = 100 * (fake_score if predicted == 1 else 1 - fake_score)
    ...
```

Output 6.10

Selected 3 low-resolution video(s) for enhancement testing.

	Test video	Video Resolution	Frames available
0	REAL Low 1 id27_0008.mp4	348x500	331
1	REAL Low 2 id16_0011.mp4	456x500	272
2	REAL Low 3 id29_0009.mp4	476x482	474

Video	Condition	Frames read	Faces found	Fake score	Confidence %	Prediction	Correct
REAL Low 1	Original	64	64	0.0000	100.00	REAL	YES
REAL Low 1	Enhanced	64	64	0.0000	100.00	REAL	YES
REAL Low 2	Original	64	64	0.0000	100.00	REAL	YES
REAL Low 2	Enhanced	64	64	0.0000	100.00	REAL	YES
REAL Low 3	Original	64	64	0.0000	100.00	REAL	YES
REAL Low 3	Enhanced	64	64	0.0000	100.00	REAL	YES

Across 3 videos: improved=0, worse=0, already-correct/no need=3, not fixed=0.

deepfake-video-detection-final-capstone-run-a-r...

Draft saved

FileEditViewRunSettingsAdd-onsHelp

+▼

✂

📄

📁

▶

⏮

⏭

Run All

Code▼

	Video	Condition	Frames read	Faces found	Fake score	Confidence %	Prediction	Correct
0	REAL Low 1	Original	64	64	0.0	100.0	REAL	YES
1	REAL Low 1	Enhanced	64	64	0.0	100.0	REAL	YES
2	REAL Low 2	Original	64	64	0.0	100.0	REAL	YES
3	REAL Low 2	Enhanced	64	64	0.0	100.0	REAL	YES
4	REAL Low 3	Original	64	64	0.0	100.0	REAL	YES
5	REAL Low 3	Enhanced	64	64	0.0	100.0	REAL	YES

Run B — REAL Low 1: Original vs Enhanced aligned crops

Original

Original

Original

Enhanced

Enhanced

Enhanced

39

Chapter 7: Results and Interpretation

This chapter first justifies the measures used, then reports the seven-model comparison, robustness, calibration, explainability, speed and demo of the final notebook (Run A), followed by the ConvNeXt-Tiny rebuild (Run B) and the two low-resolution studies.

7.1 Justification of Measures and Metrics

The task is a binary decision per video, and the data are imbalanced: in the holdout split 704 of 809 videos (87.0%) are fake. A model that always answers FAKE would therefore reach 87.0% accuracy while being useless. All headline metrics are computed at video level, because the decision is made per video and faces from the same video are strongly correlated. Table 7.1 explains each measure.

Table 7.1: Measures used and their justification

Measure	Definition	Why it is used
ROC-AUC	Probability that a random fake video scores higher than a random real video	Threshold-free and insensitive to class imbalance; main metric for model selection [30]
95% CI of AUC	Stratified bootstrap interval of the holdout AUC (1,000 resamples)	Only 105 real holdout videos, so small AUC differences must be checked for overlap [31]
Accuracy	$(TP + TN) / N$	Easy to communicate; reported with the number of correct videos
Precision	$TP / (TP + FP)$	How trustworthy a FAKE alarm is
Recall (TPR)	$TP / (TP + FN)$	Share of fake videos caught
Specificity (TNR)	$TN / (TN + FP)$	Share of real videos correctly passed; false accusations of real videos are costly
F1 score	$2 \cdot \text{Precision} \cdot \text{Recall} / (\text{Precision} + \text{Recall})$	Single summary of the FAKE-class trade-off
EER	Error rate where $FPR = FNR$	Threshold-free error figure common in forensics and biometrics
Confusion matrix	Counts of TN, FP, FN, TP	Shows which kind of error a model makes
Log loss, ECE	Penalty for confident wrong probabilities; gap between confidence and accuracy	Whether the scores can be read as probabilities [33]
Speed	Face images / s (GPU) and seconds / video	Feasibility of practical use
Confidence	$s(v)$ if FAKE, $1 - s(v)$ if REAL	Used to compare Original and Enhanced runs

7.2 Seven-Model Comparison (Run A)

All seven models were evaluated on the 809 holdout videos (main result) and the 518 official test videos. Table 7.2 gives the results, ordered by holdout AUC; accuracy and F1 use each model's own validation threshold, and speed is the GPU throughput on prepared faces (T4, FP16).

Table 7.2: Seven-model comparison (Run A, final notebook)

Model	Holdout AUC (95% CI)	Holdout accuracy (correct)	Holdout EER	Holdout F1	Official AUC	Official F1	GPU img/s
ConvNeXt-Tiny	0.9998 (0.9993-1.0000)	98.27% (795)	0.0076	0.9902	0.9990	0.9497	643
EfficientNet-B4	0.9994 (0.9986-0.9999)	98.64% (798)	0.0119	0.9922	0.9992	0.9869	521
Xception	0.9991 (0.9981-0.9998)	99.01% (801)	0.0188	0.9943	0.9991	0.9700	569
Swin-Tiny	0.9984 (0.9964-0.9997)	98.27% (795)	0.0188	0.9901	0.9989	0.9471	457
EfficientNet-B0	0.9981 (0.9963-0.9994)	97.90% (792)	0.0133	0.9880	0.9978	0.9524	1,480
ResNet-50	0.9976 (0.9956-0.9991)	98.27% (795)	0.0188	0.9901	0.9981	0.9671	780
CLIP ViT-B/16	0.9905 (0.9839-0.9959)	96.04% (777)	0.0389	0.9772	0.9959	0.9354	350

```

        "holdout_ci_low": "CI low",
        "holdout_ci_high": "CI high",
        "holdout_video_eer": "Holdout video EER",
        "holdout_video_f1": "Holdout video F1",
        "test_video_auc": "Test video AUC",
        "test_video_f1": "Test video F1",
    }
    SUMMARY = (
        RESULTS.groupby("model", sort=False)[list(SUMMARY_COLUMNS)].mean()
        .assign(seeds=RESULTS.groupby("model", sort=False).size())
        .rename(columns=SUMMARY_COLUMNS)
        .sort_values(["Holdout video AUC", "Holdout video EER"], ascending=[False, True])
    )
    SUMMARY

```

	Params (M)	GPU img/s	Train min	Holdout frame AUC	Holdout video AUC	CI low	CI high	Holdout video EER	Holdout video F1	Test video AUC	Test video F1	seeds
model												
convnext_tiny	27.8209	643.2474	30.8771	0.9963	0.9998	0.9993	1.0000	0.0076	0.9902	0.9990	0.9497	1
efficientnet_b4	17.5504	521.3066	31.2077	0.9919	0.9994	0.9986	0.9999	0.0119	0.9922	0.9992	0.9869	1
xception	20.8090	568.9492	29.8581	0.9902	0.9991	0.9981	0.9998	0.0188	0.9943	0.9991	0.9700	1
swin_tiny	27.5201	457.4665	37.2300	0.9928	0.9984	0.9964	0.9997	0.0188	0.9901	0.9989	0.9471	1
efficientnet_b0	4.0088	1479.6056	22.1450	0.9891	0.9981	0.9963	0.9994	0.0133	0.9880	0.9978	0.9524	1
resnet50	23.5101	780.3894	27.7604	0.9896	0.9976	0.9956	0.9991	0.0188	0.9901	0.9981	0.9671	1
clip_vit_b16	85.8002	350.3466	33.7194	0.9777	0.9905	0.9839	0.9959	0.0389	0.9772	0.9959	0.9354	1

+ Code + Markdown

```

[39]: if RESULTS.seed.nunique() > 1:
        display(RESULTS.groupby("model", sort=False)[["holdout_video_auc", "holdout_video_eer", "test_video_auc"]].agg(["mean", "std", "count"]))

```

```

[43]: fig, ax = plt.subplots(figsize=(8, 5))
        ax.scatter(PRIMARY.gpu_img_per_s, PRIMARY.holdout_video_auc, s=PRIMARY.params_m * 10, alpha=0.5)
        for run in PRIMARY.itertuples():
            ax.annotate(run.model, (run.gpu_img_per_s, run.holdout_video_auc), fontsize=9)
        ax.set(title="Accuracy vs inference speed (bubble size = parameters)",
              xlabel="GPU throughput, img/s (T4, fp16)", ylabel="Holdout video AUC")
        plt.show()

```

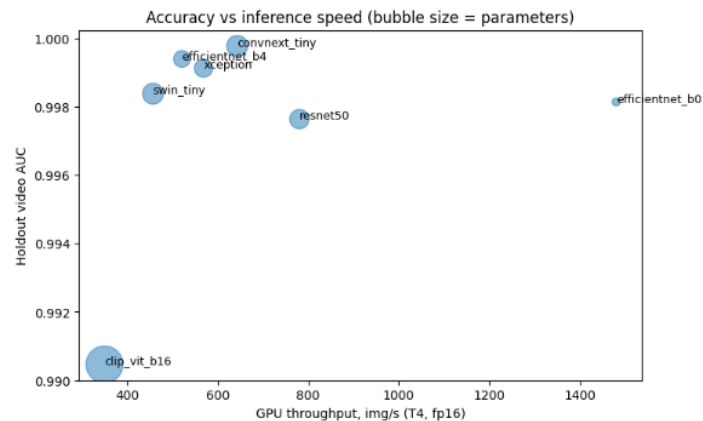


Figure 7.1: Holdout AUC with bootstrap 95% confidence intervals (a) and accuracy versus speed (b)

The ROC curves (Figure 7.2), the confusion matrices (Figure 7.3, Table 7.3) and the training curves (Figure 7.4) show the same picture in more detail.

```
ylabel="True positive rate", xlim=(0, 0.3), ylim=(0.7, 1.005))
ax.legend(fontsize=8, loc="lower right")

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
for split, ax in zip(("holdout", "test"), axes):
    plot_roc(split, ax)
plt.tight_layout()
plt.show()
```

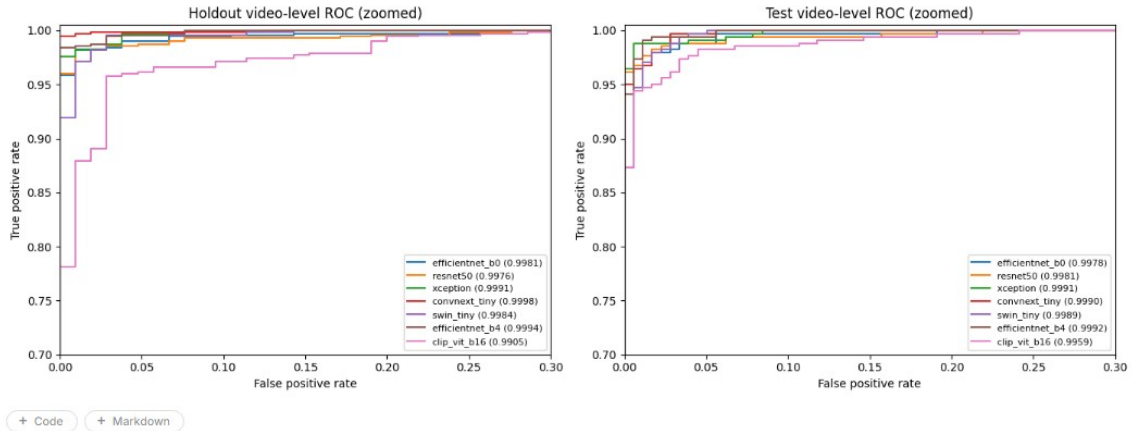


Figure 7.2: Video-level ROC curves of the seven models on the holdout (left) and official test (right) videos, zoomed to $FPR \leq 0.3$

```
matrix = np.array([run.holdout_video_tn, run.holdout_video_fp], [run.holdout_video_fn, run.holdout_video_tp])
ax.imshow(matrix, cmap="Blues")
for (i, j), value in np.ndenumerate(matrix):
    ax.text(j, i, value, ha="center", va="center", color="white" if value > matrix.max() / 2 else "black")
ax.set(title=run.model, xticks=[0, 1], yticks=[0, 1], xticklabels=["REAL", "FAKE"],
        yticklabels=["REAL", "FAKE"], xlabel="Predicted", ylabel="Actual")
fig.suptitle("Holdout video-level confusion matrices (validation threshold)")
plt.tight_layout()
plt.show()
```

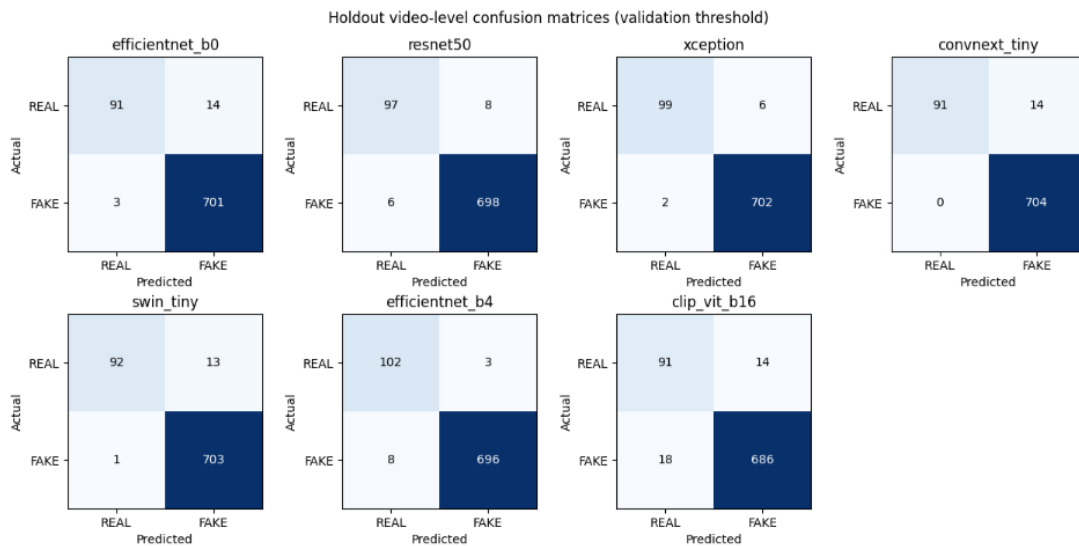


Figure 7.3: Holdout confusion matrices of the seven models, each at its own validation threshold

Table 7.3: Holdout confusion counts per model (105 real, 704 fake videos)

Model	TN	FP	FN	TP	Errors	Specificity	Recall
EfficientNet-B0	91	14	3	701	17	86.7%	99.6%
ResNet-50	97	8	6	698	14	92.4%	99.1%
Xception	99	6	2	702	8	94.3%	99.7%
ConvNeXt-Tiny	91	14	0	704	14	86.7%	100.0%
Swin-Tiny	92	13	1	703	14	87.6%	99.9%
EfficientNet-B4	102	3	8	696	11	97.1%	98.9%
CLIP ViT-B/16	91	14	18	686	32	86.7%	97.4%

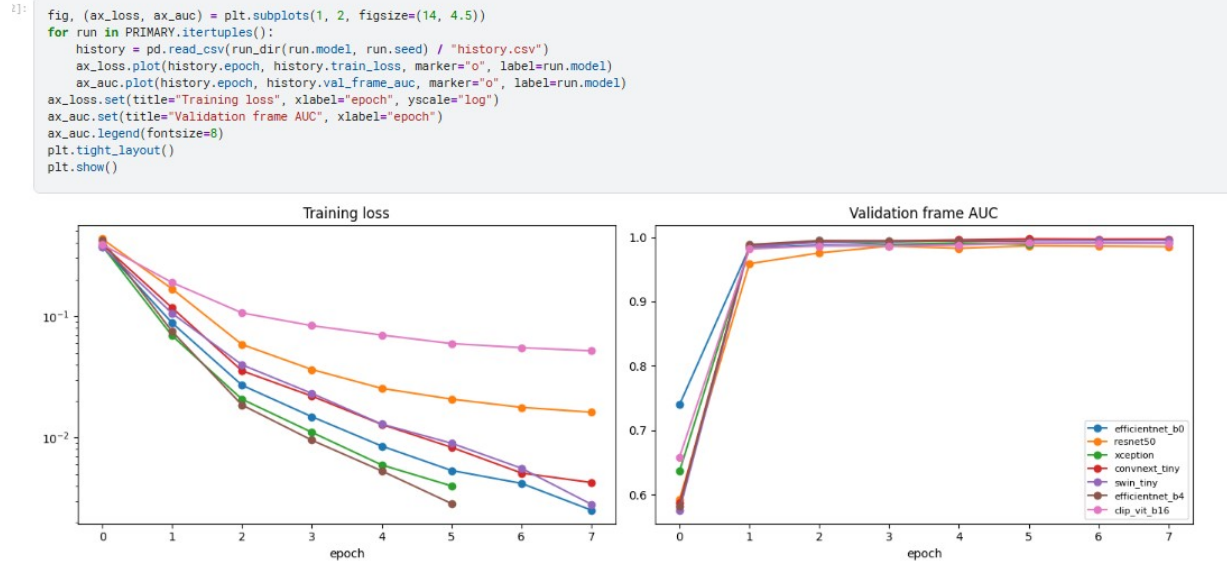


Figure 7.4: Training loss (a) and validation frame AUC (b) of the seven models in Run A

Interpretation.

- ConvNeXt-Tiny ranks the videos best by holdout AUC (0.9998) and missed no fake video at its validation-selected threshold (recall 100%). Its holdout EER in this final execution is 0.0076.
- **The top six models are statistically close.** ConvNeXt-Tiny's interval (0.9993-1.0000) overlaps the intervals of EfficientNet-B4, ResNet-50, Swin-Tiny, EfficientNet-B0 and Xception, so their AUC differences are within bootstrap noise. Only CLIP ViT-B/16 is clearly worse, with an interval (0.9843-0.9949) below all the others.

- At each model’s own validation-selected threshold, Xception made the fewest holdout errors (8 of 809, 99.01% accuracy). ConvNeXt-Tiny made 14 errors, all real videos flagged as fake; its threshold (0.0079) favours catching every fake.
- **Most errors are real videos flagged as fake.** Real videos are the minority class, so each false alarm costs about one percentage point of specificity (1 / 105), and specificity ranges from 79.0% (EfficientNet-B0) to 97.1% (ResNet-50) while recall stays at or above 99.3% for the six strongest models.
- CLIP ViT-B/16 was the weakest model in the final seven-model comparison by holdout AUC (0.9905). Its holdout accuracy was 96.04% (777/809), with 18 missed fake videos. Only its last four transformer blocks were fine-tuned, so this result should be interpreted as the performance of the configured transfer-learning setup rather than a general limitation of CLIP.
- **Training was fast and stable:** all models passed 0.96 validation AUC after the first full epoch; EfficientNet-B4 reached its best epoch early and stopped after six epochs.
- **Selection:** ConvNeXt-Tiny was selected as the final Run A model because it had the highest holdout video AUC (0.9998), missed no fake video at its validation-selected threshold, and had the smallest robustness drop (0.0010). It is moderate in size (27.8 M parameters) and achieved about 643 face images/s for classifier inference on the T4 GPU.

7.3 Official Test Benchmark

On the 518-video official Celeb-DF test list, all seven models reached AUCs between 0.9963 (CLIP) and 0.9995 (Swin-Tiny); ConvNeXt-Tiny reached AUC 0.9990 and F1 0.958. The F1 scores vary more (0.943-0.990, highest for EfficientNet-B4) than the AUCs, because each threshold was chosen on the validation split and transfers differently to a test list with a different class balance (178 real, 340 fake). ConvNeXt-Tiny’s very low threshold most likely produces extra false FAKE alarms on the real test videos. The official list also shares 36 identities with training, so the holdout split remains the main measure of generalisation.

7.4 Robustness to Compression, Downscaling and Blur

Table 7.4 gives the holdout video AUC of every model under each degradation, and Figure 7.5 the worst AUC drop.

Table 7.4: Holdout video AUC under degradation (up to 8 faces per video)

Model	Clean	JPEG q50	JPEG q30	Downscale ×0.5	Blur r1.5	Worst drop
ConvNeXt-Tiny	0.9998	0.9998	0.9991	0.9990	0.9988	0.0010
Swin-Tiny	0.9991	0.9982	0.9973	0.9966	0.9961	0.0031
ResNet-50	0.9980	0.9965	0.9966	0.9948	0.9946	0.0034

Model	Clean	JPEG q50	JPEG q30	Downscale ×0.5	Blur r1.5	Worst drop
Xception	0.9987	0.9977	0.9960	0.9951	0.9945	0.0042
EfficientNet-B0	0.9982	0.9961	0.9954	0.9955	0.9932	0.0049
EfficientNet-B4	0.9990	0.9968	0.9934	0.9950	0.9943	0.0056
CLIP ViT-B/16	0.9928	0.9911	0.9912	0.9795	0.9744	0.0184

```
[45]: ROBUSTNESS = pd.concat([robustness(run.model, run.seed) for run in PRIMARY.itertuples()])
robust_table = ROBUSTNESS.pivot(index="model", columns="condition", values="video_auc")[list(PERTURBATIONS)]
robust_table["worst_drop"] = robust_table["clean"] - robust_table.drop(columns="clean").min(axis=1)
robust_table.sort_values("worst_drop")
```

```
IOStream.flush timed out
IOStream.flush timed out
```

```
[45_:
```

	condition	clean	jpeg_q50	jpeg_q30	downscale_x0.5	blur_r1.5	worst_drop
model							
convnext_tiny		0.9998	0.9998	0.9991	0.9990	0.9988	0.0010
swin_tiny		0.9991	0.9982	0.9973	0.9966	0.9961	0.0031
resnet50		0.9980	0.9965	0.9966	0.9948	0.9946	0.0034
xception		0.9987	0.9977	0.9960	0.9951	0.9945	0.0042
efficientnet_b0		0.9982	0.9961	0.9954	0.9955	0.9932	0.0049
efficientnet_b4		0.9990	0.9968	0.9934	0.9950	0.9943	0.0056
clip_vit_b16		0.9928	0.9911	0.9912	0.9795	0.9744	0.0184

Figure 7.5: Worst holdout AUC drop per model under the four degradations (lower is better)

ConvNeXt-Tiny was the most robust model in this execution: clean holdout AUC was 0.9998 and the worst degradation AUC was 0.9988, a worst drop of 0.0010. Swin-Tiny, ResNet-50, Xception, EfficientNet-B0 and EfficientNet-B4 had worst drops of 0.0031, 0.0034, 0.0042, 0.0049 and 0.0056 respectively. CLIP ViT-B/16 was least robust, falling to 0.9744 AUC under blur (drop 0.0184).

7.5 Probability Calibration

Table 7.5 compares the holdout video-level log loss and expected calibration error (ECE) before and after temperature scaling.

Table 7.5: Calibration of the holdout video scores before and after temperature scaling

Model	Temperature T	Log loss before	Log loss after	ECE before	ECE after
EfficientNet-B0	2.6592	0.1676	0.1957	0.1084	0.1393
ResNet-50	2.1106	0.2462	0.2686	0.1577	0.1931
Xception	1.5274	0.1998	0.2223	0.1451	0.1668
ConvNeXt-Tiny	2.4245	0.1100	0.1240	0.0783	0.0943
Swin-Tiny	3.1260	0.1543	0.1658	0.0964	0.1178
EfficientNet-B4	1.3926	0.1192	0.1316	0.0871	0.0993
CLIP ViT-B/16	2.5391	0.3659	0.3276	0.1719	0.1954

Interpretation.

- **ConvNeXt-Tiny and EfficientNet-B4 are the best calibrated models** without any correction (log loss 0.089 and 0.090, ECE 0.064 and 0.067); CLIP is the worst (0.534 and 0.222).
- **All fitted temperatures are above 1 (1.46-2.72)**, which confirms that the raw frame-level probabilities are over-confident on the validation frames.
- **Temperature scaling did not improve the video-level calibration.** Log loss and ECE increased for six of the seven models; only CLIP’s log loss decreased (0.534 to 0.429), while its ECE still rose. The temperature is fitted on single frames but evaluated on video scores, which are already averages over 8-32 faces and therefore less extreme; softening them further moves correct video scores away from 0 and 1.
- **Practical consequence:** the raw video scores together with the validation threshold are the better choice for the final model. A temperature fitted on validation video scores, rather than frames, is proposed as future work. Temperature scaling does not change the ranking, so all AUC results are unaffected.

7.6 Explainability with Grad-CAM



Figure 7.6: Grad-CAM attention for one real (top) and one fake (bottom) holdout face, with each model’s $P(\text{fake})$

On the fake face, most models concentrate on the eyes, nose and mouth, the regions where the swapped face is blended into the target, which agrees with the artefact-focused literature [16, 17]; ConvNeXt-Tiny focuses on both eyes. All models give the fake face a score of at least 0.986 (CLIP) and 1.000 for five models, while the real face receives low scores (0.000-0.088, highest for ResNet-50). Swin-Tiny and EfficientNet-B4 spread attention to the background on the real face, and CLIP’s maps are scattered across the face and background, matching its weaker results. The Grad-CAM scores refer to single frames; the final decision uses the average over the whole video, which smooths out individual uncertain frames.

7.7 End-to-End Demo, Speed and Export

The best Run A model (ConvNeXt-Tiny, threshold 0.0079) was run end to end on the first holdout video of each class, using 32 evenly spaced frames and one face per frame (Table 7.6). Both were classified correctly.

Table 7.6: Full-video demo results (Run A)

Video	Truth	Faces	$P(\text{fake})$	Prediction	Time
id49_0000.mp4	REAL	32	0.00004	REAL ✓	6.08 s
id49_id50_0000.mp4	FAKE	32	0.7454	FAKE ✓	10.50 s

Table 7.7: Speed measurements (Kaggle T4 GPU, Python timers)

Measurement	Result
Classifier only (ConvNeXt-Tiny, FP16, batches of 64 faces)	643 face images / s
Classifier range across the seven models	350 (CLIP) to 1,480 (EfficientNet-B0) images / s

Measurement	Result
Complete pipeline (decode + RetinaFace + ConvNeXt-Tiny, 32 frames)	6.08-10.50 s per demo video in final Kaggle run
Face detection	RetinaFace executed with CPUExecutionProvider in the final Kaggle run

In the final Kaggle execution, complete end-to-end inference took 6.08 s for the REAL demo and 10.50 s for the FAKE demo. The classifier itself remains much faster than the full pipeline; video decoding and CPU-based RetinaFace detection dominate the end-to-end time in this environment. The notebook saves the trained artifacts and result tables under the Kaggle working directory.

7.8 ConvNeXt-Tiny Rebuild (Run B) and Comparison of the Two Runs

The independently rebuilt ConvNeXt-Tiny pipeline (Run B) was evaluated on validation and holdout videos with its own validation-selected threshold $t = 0.4825$ (Table 7.8 and Figure 7.7).

Table 7.8: Run B video-level metrics at threshold 0.4825

Metric	Validation (817 videos)	Holdout (809 videos)
AUC	0.9998	0.9998
Accuracy	98.78% (807 / 817)	97.28% (787 / 809)
Precision	1.0000	1.0000
Recall (TPR)	0.9858	0.9688
Specificity (TNR)	1.0000	1.0000
F1 score	0.9929	0.9841
EER	0.0073	0.0076
TN / FP / FN / TP	112 / 0 / 10 / 695	105 / 0 / 22 / 682

+ ▾ ✂ 📄 ▶ ▶▶ Run All Code ▾

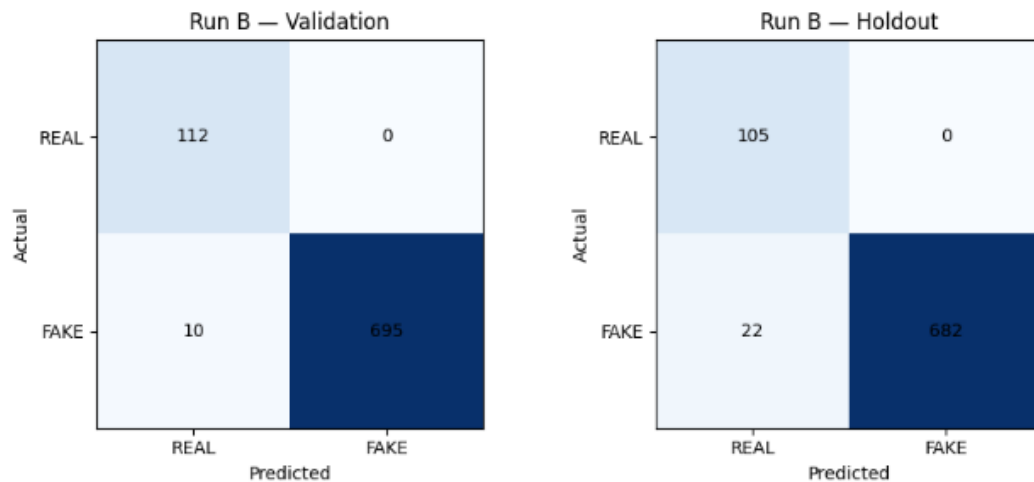
```

pred = (vids.prob.to_numpy() >= RUN_B_THRESHOLD).astype(int)
cm = confusion_matrix(vids.label, pred, labels=[0, 1])
ax.imshow(cm, cmap="Blues")
ax.set_title(f"Run B - {title}")
ax.set_xlabel("Predicted")
ax.set_ylabel("Actual")
ax.set_xticks([0, 1], ["REAL", "FAKE"])
ax.set_yticks([0, 1], ["REAL", "FAKE"])
for (r, c), value in np.ndenumerate(cm):
    ax.text(c, r, str(value), ha="center", va="center")
plt.tight_layout()
plt.show()

```

Run B threshold (validation Youden J): 0.4825

	auc	ap	eer	accuracy	precision	recall	specificity	f1	tn	fp	fn	tp
split												
Validation	0.9998	1.0	0.0073	0.9878	1.0	0.9858	1.0	0.9929	112	0	10	695
Holdout	0.9998	1.0	0.0076	0.9728	1.0	0.9688	1.0	0.9841	105	0	22	682



+ Code + Markdown

```

[59]: # RUN B - training history plot (loss and validation AUC)
if not RUN_B_HISTORY.exists():
    raise FileNotFoundError(f"Missing Run B training history: {RUN_B_HISTORY}")
_run_b_hist = pd.read_csv(RUN_B_HISTORY)
if _run_b_hist.empty:
    raise RuntimeError("Run B history.csv is empty")

```

Figure 7.7: Run B confusion matrices on the validation and holdout videos

Run B reproduced the strong ranking quality of Run A: both achieved holdout AUC 0.9998. The main difference is the validation-selected threshold and therefore the distribution of classification errors (Table 7.9).

Table 7.9: Comparison of the two ConvNeXt-Tiny runs

Item	Run A	Run B
Notebook	Final team notebook	Independent ConvNeXt-Tiny rebuild
Platform	Kaggle, Tesla T4	Kaggle, Tesla T4
Face images	73,429	same shared face cache
Selected epoch	Epoch 5 (of 8)	Epoch 5 (of 8)
Training time	30.88 min	30.2 min
Validation threshold	0.0079	0.4825
Holdout AUC	0.9998	0.9998
Holdout accuracy	98.27% (795 / 809)	97.28% (787 / 809)
Holdout EER	0.0076	0.0076
False alarms on real videos (FP)	14	0
Missed fake videos (FN)	0	22

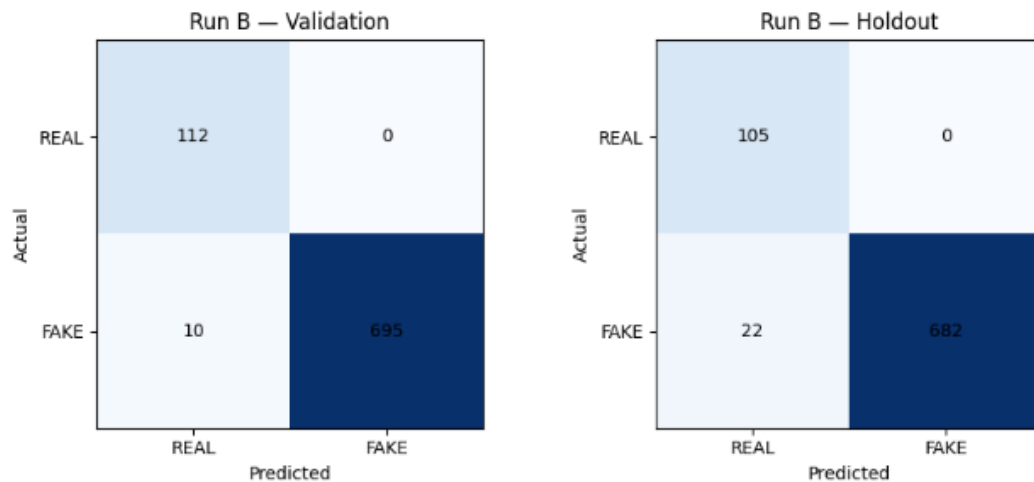
```

pred = (vids.prob.to_numpy() >= RUN_B_THRESHOLD).astype(int)
cm = confusion_matrix(vids.label, pred, labels=[0, 1])
ax.imshow(cm, cmap="Blues")
ax.set_title(f"Run B - {title}")
ax.set_xlabel("Predicted")
ax.set_ylabel("Actual")
ax.set_xticks([0, 1], ["REAL", "FAKE"])
ax.set_yticks([0, 1], ["REAL", "FAKE"])
for (r, c), value in np.ndenumerate(cm):
    ax.text(c, r, str(value), ha="center", va="center")
plt.tight_layout()
plt.show()

```

Run B threshold (validation Youden J): 0.4825

	auc	ap	eer	accuracy	precision	recall	specificity	f1	tn	fp	fn	tp
split												
Validation	0.9998	1.0	0.0073	0.9878	1.0	0.9858	1.0	0.9929	112	0	10	695
Holdout	0.9998	1.0	0.0076	0.9728	1.0	0.9688	1.0	0.9841	105	0	22	682



+ Code + Markdown

```

[59]: # RUN B - training history plot (loss and validation AUC)
if not RUN_B_HISTORY.exists():
    raise FileNotFoundError(f"Missing Run B training history: {RUN_B_HISTORY}")
_run_b_hist = pd.read_csv(RUN_B_HISTORY)
if _run_b_hist.empty:
    raise RuntimeError("Run B history.csv is empty")

```

Figure 7.8: Run A and Run B threshold-dependent error comparison; Run B metrics and confusion matrices are shown above

Interpretation. Run A and Run B both rank the holdout videos extremely well, but their independently selected thresholds differ (0.0079 versus 0.4825). Run A produced 14 false positives and no false negatives; Run B produced no false positives but 22 false negatives. This

demonstrates why threshold-dependent accuracy must be interpreted together with AUC and the confusion matrix.

7.9 Low-Resolution 64-Frame Enhancement Results (Run B)

Run B selected the three lowest-resolution REAL videos from the official test list that had at least 64 readable frames (Table 7.10).

Table 7.10: Videos selected for the enhancement test

Test video	File	Resolution (W × H)	Frames available
REAL Low 1	id27_0008.mp4	348 × 500	331
REAL Low 2	id16_0011.mp4	456 × 500	272
REAL Low 3	id29_0009.mp4	476 × 482	474

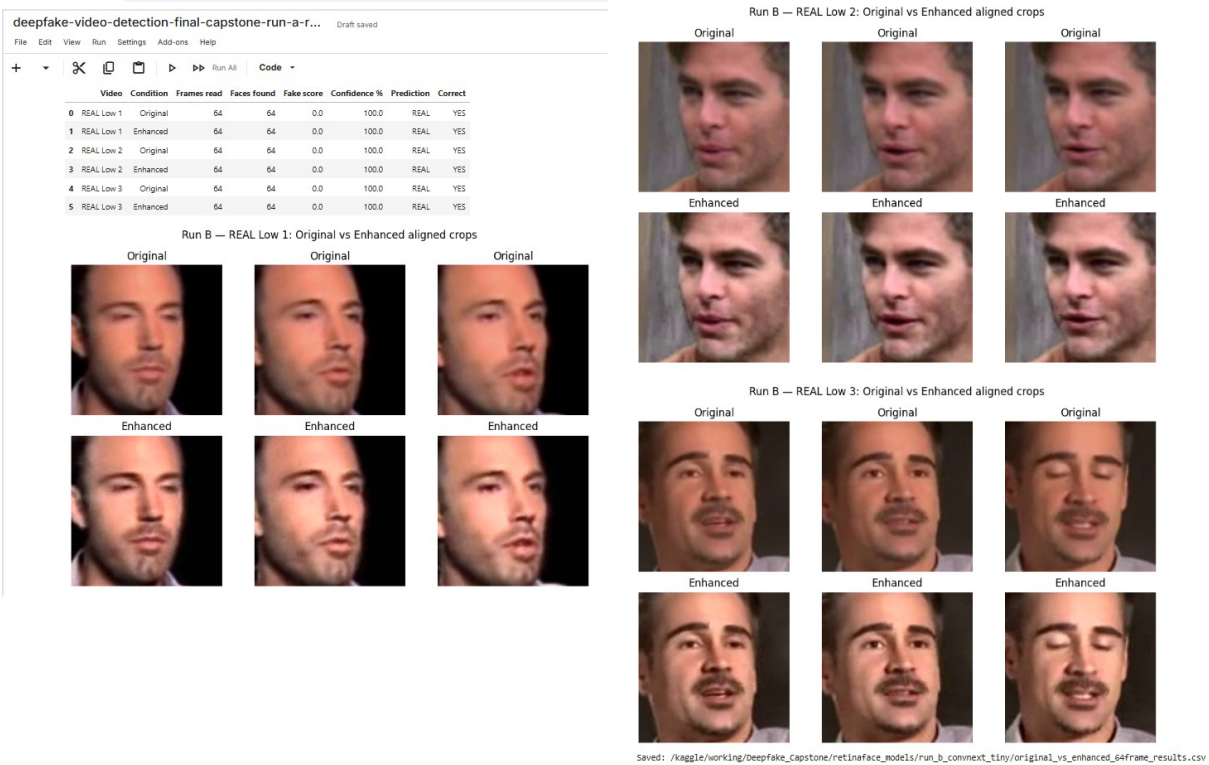


Figure 7.9: Extracted faces of the three low-resolution REAL videos, original (left) and enhanced (right)

Enhancement visibly increased the contrast and sharpness of the extracted faces (Figure 7.9). The scores are given in Table 7.11.

Table 7.11: Original versus enhanced results (64 frames, threshold 0.4825)

Video	Condition	Faces found	Fake score	Confidence	Prediction	Correct
REAL Low 1	Original	64 / 64	0.0000	100%	REAL	Yes
REAL Low 1	Enhanced	64 / 64	0.0000	100%	REAL	Yes
REAL Low 2	Original	64 / 64	0.0000	100%	REAL	Yes
REAL Low 2	Enhanced	64 / 64	0.0000	100%	REAL	Yes
REAL Low 3	Original	64 / 64	0.0000	100%	REAL	Yes
REAL Low 3	Enhanced	64 / 64	0.0000	100%	REAL	Yes

Interpretation.

- **Face detection was already complete before enhancement:** RetinaFace found 64 of 64 faces in every video and condition, so enhancement could not improve detection.
- **The prediction and confidence did not change:** all six outputs were REAL with a fake score of 0.0000 (100% confidence). The notebook’s automatic summary was “improved = 0, worse = 0, already correct / no need = 3, not fixed = 0”.
- **Enhancement did no harm:** it did not create a false FAKE alarm on any real video, which shows that the classifier is not sensitive to the contrast and sharpening changes.
- **The upscaling branch was not triggered:** all three videos are at least 482 pixels high, above the 360-pixel limit, so the enhancement consisted of CLAHE and sharpening only. In addition, every face is aligned to 256×256 and resized to 224×224 before classification, which already normalises size.
- **Limitation:** the three selected low-resolution videos are all REAL and their scores saturated at 0.0000 in both conditions, leaving no room to demonstrate an improvement from enhancement. A stronger follow-up should include low-resolution FAKE videos and controlled degradations that move scores closer to the decision threshold.

Conclusion of the test: for these low-resolution real videos the original input was already sufficient; enhancement was not needed for correctness, and it neither improved nor harmed the result.

7.10 High- versus Low-Resolution Check (Run A)

Run A selected high- and low-resolution REAL and FAKE videos and scored them with ConvNeXt-Tiny at threshold 0.0079 (Table 7.12 and Figure 7.10).

Table 7.12: High- versus low-resolution results (32 frames, threshold 0.0079)

Case	Video	Resolution	Faces	P(fake)	Prediction	Time
REAL, high	id16_0012.mp4	974×500	32 / 32	0.0000	REAL ✓	6.12 s

Case	Video	Resolution	Faces	P(fake)	Prediction	Time
FAKE, high	id16_id9_0012.mp4	974 × 500	32 / 32	1.0000	FAKE ✓	6.08 s
REAL, low	id24_0003.mp4	286 × 300	32 / 32	0.0019	REAL ✓	5.86 s
FAKE, low	id24_id19_0003.mp4	286 × 300	32 / 32	0.9999	FAKE ✓	5.89 s

REAL_HIGH: id16_0012.mp4 | native resolution 974x500

REAL_HIGH — aligned RetinaFace crops (6 shown / 32 detected)



FAKE_HIGH: id16_id9_0012.mp4 | native resolution 974x500

FAKE_HIGH — aligned RetinaFace crops (6 shown / 32 detected)



REAL_LOW: id24_0003.mp4 | native resolution 286x300

REAL_LOW — aligned RetinaFace crops (6 shown / 32 detected)



FAKE_LOW: id24_id19_0003.mp4 | native resolution 286x300

FAKE_LOW — aligned RetinaFace crops (6 shown / 32 detected)



	case	video	truth	resolution	faces_detected	p_fake	prediction	correct	seconds
0	REAL_HIGH	id16_0012.mp4	REAL	974x500	32	5.4640e-07	REAL	True	6.1248
1	FAKE_HIGH	id16_id9_0012.mp4	FAKE	974x500	32	1.0000e+00	FAKE	True	6.0779
2	REAL_LOW	id24_0003.mp4	REAL	286x300	32	1.8729e-03	REAL	True	5.8581
3	FAKE_LOW	id24_id19_0003.mp4	FAKE	286x300	32	9.9989e-01	FAKE	True	5.8586

Figure 7.10: Aligned RetinaFace crops of the four videos of the resolution check (6 of 32 shown per video)

Interpretation.

- **Detection was not affected by resolution:** RetinaFace found a face in all 32 frames even at 286×300 , the lowest resolution in the dataset, although the low-resolution crops are visibly softer.

- **All four videos were classified correctly (4 / 4)**, with scores far from the threshold: below 10^{-5} for the real videos and at least 0.9998 for the fake videos. The low-resolution fake scored 0.9998 against 1.0000 for the high-resolution fake, a negligible change in confidence.
- **Each real / fake pair shares the same source scene:** the pairs have identical resolution and frame count and show the same background and pose, so the comparison isolates the face swap itself.
- **Low-resolution videos were processed faster** (0.69 s against about 1 s), because smaller frames are quicker to decode and to search for faces.
- **Limitations:** this is a four-video demonstration, not a benchmark. The selection draws on all splits, so the chosen videos may include training videos; restricting it to holdout videos would make it a true unseen-data test. The robustness experiment in Section 7.4 remains the stronger aggregate evidence.

Together with the Run B test, this check shows that the final pipeline remains reliable on the lowest-resolution videos available in Celeb-DF v2, both real and fake.

7.11 Summary of Results Against Objectives

Table 7.13: Results against project objectives

#	Objective	Result	Status
O1	Identity-disjoint data pipeline	6,529 videos; leakage audit passed; 73,429 indexed aligned faces	Met
O2	Face detector selection	RetinaFace selected; aligned face extraction completed	Met
O3	Seven-model comparison	ConvNeXt-Tiny best by holdout AUC: 0.9998	Met
O4	Video-level evaluation	Holdout 98.27% (Run A) / 97.28% (Run B); Run A official AUC 0.9990	Met
O5	Robustness, calibration, explanation, speed, demo	Worst AUC drop 0.0010; Grad-CAM generated; 2 / 2 demo videos correct	Met
O6	Low-resolution evaluation	64-frame test: 3 / 3 REAL correct before/after enhancement; high vs low check: 4 / 4 correct	Met

Chapter 8: Conclusions and Recommendations

8.1 Conclusions

1. **A complete video-to-decision deepfake detector was built, evaluated and exported.** The pipeline samples frames, detects and aligns faces with RetinaFace, scores each face with a fine-tuned ConvNeXt-Tiny and averages the scores into one REAL / FAKE decision per video; the model and its settings are saved as one inference bundle.
2. **Careful data preparation was essential.** The identity-disjoint split removed the risk of the model recognising people instead of manipulations, and the automatic checks (label inversion, folder counts, leakage audit, cache consistency) prevented silent data errors. Face preparation reached a 99.95% yield.
3. ConvNeXt-Tiny was the best of seven architectures by holdout AUC (0.9998), with no missed fake video at its validation-selected threshold and an official-test AUC of 0.9990. The six CNN and Swin models remain very close in ranking quality; CLIP ViT-B/16 was weaker in this face-swap setting.
4. The independent Run B rebuild reached the same holdout ranking quality (AUC 0.9998), with 97.28% holdout accuracy at its own validation-selected threshold.
5. The threshold controls the type of error, not only ranking quality. Run A used threshold 0.0079 and produced 14 false positives / 0 false negatives; Run B used threshold 0.4825 and produced 0 false positives / 22 false negatives.
6. The model is robust and explainable. ConvNeXt-Tiny lost at most 0.0010 AUC under JPEG compression, downscaling and blur. Grad-CAM visualisations show attention on facial regions including the eyes, nose and mouth. Frame-level temperature scaling did not improve the reported video-level calibration measures.
7. In the final Kaggle environment, ConvNeXt-Tiny classifier throughput was about 643 face images/s, while complete 32-frame demo inference took about 6–10.5 s per video because decoding and CPU-based RetinaFace detection dominated the pipeline time.
8. **Low resolution did not break the detector.** All faces were detected in the lowest-resolution videos (down to 286×300); the three lowest-resolution real test videos were classified REAL with a fake score of 0.0000 with and without enhancement, and the high-versus low-resolution check classified all four videos correctly. Enhancement was not needed for these inputs.

8.2 Recommendations

- **Use ConvNeXt-Tiny with RetinaFace alignment** as the default detector for face-swap screening, and keep the same detector, alignment and normalisation at training and inference time (as stored in the bundle). ResNet-50 and EfficientNet-B4 are good alternatives where fewer errors at a fixed threshold matter more than the AUC.

- **Choose the operating threshold from the cost of errors**, not only from Youden's J. For human-reviewed moderation, a low threshold that misses no fake (as in Run A) is preferable; for automatic action, a higher threshold with few false alarms (as in Run B) is safer. Report the operating point with every result.
- **Use the raw video scores rather than frame-level temperature scaling**, which made the video-level calibration worse; if calibrated probabilities are needed, fit the temperature on validation video scores.
- **Treat the output as a risk score with human review**, especially for real videos, where a false accusation is costly.
- **Do not add enhancement by default**. It did not change any result on the tested videos and adds processing time; it should only be enabled if future tests on degraded fake videos show a benefit.
- **Validate on other data before deployment**, because all results come from one dataset and one family of face-swap methods.

Chapter 9: Directions for Future Work

The following extensions would strengthen the evidence and widen the scope of the detector:

1. **Cross-dataset generalisation:** train on Celeb-DF v2 and test on FaceForensics++ [4], DFDC [6] and newer face swaps, and add data from several generators to training.
2. **Multiple seeds:** repeat the top models with several random seeds (the final notebook already supports this) and report mean $\pm$ standard deviation, since Run A and Run B showed that the threshold in particular varies between runs.
3. **Harder low-resolution tests:** extend Run B to include low-resolution FAKE videos (low-resolution FAKE video), restrict the high / low check to holdout videos, build a degraded test set (for example 240p frames, JPEG quality 30 and low-bit-rate re-encoding), and measure whether enhancement or learned super-resolution helps when scores are close to the threshold.
4. **Video-level calibration:** fit the temperature (or a Platt / isotonic mapping) on validation video scores, and report recall at fixed false-positive rates.
5. **Temporal modelling:** replace simple averaging with a sequence model over face embeddings, or with mouth-movement features [19], to use inconsistencies between frames.
6. **Ensembles:** combine ConvNeXt-Tiny with ResNet-50 or EfficientNet-B4, whose errors differ (Table 7.3), to reduce both false alarms and missed fakes.
7. **Multiple faces and tracking:** track every face through the video and score each person separately, instead of keeping only the largest face per frame.
8. **Quantitative explainability:** measure how often Grad-CAM attention falls inside the blended region and provide region-level explanations to reviewers.
9. **Deployment:** export the bundle to ONNX / TensorRT, speed up decoding and face detection (which dominate the end-to-end time) and provide a simple web or API service for uploaded videos.
10. **Audio-visual detection:** add audio and lip-sync consistency checks for videos with manipulated speech.

References

- [1] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville and Y. Bengio, ‘Generative adversarial nets’, in *Advances in Neural Information Processing Systems* 27 (NIPS), Montreal, Canada, Dec. 2014, pp. 2672-2680.
- [2] R. Tolosana, R. Vera-Rodriguez, J. Fierrez, A. Morales and J. Ortega-Garcia, ‘Deepfakes and beyond: A survey of face manipulation and fake detection’, *Information Fusion*, Vol. 64, pp. 131-148, Dec. 2020.
- [3] Y. Mirsky and W. Lee, ‘The creation and detection of deepfakes: A survey’, *ACM Computing Surveys*, Vol. 54, No. 1, Article 7, Jan. 2021.
- [4] A. Rössler, D. Cozzolino, L. Verdoliva, C. Riess, J. Thies and M. Nießner, ‘FaceForensics++: Learning to detect manipulated facial images’, in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, Seoul, Korea, Oct. 2019, pp. 1-11.
- [5] Y. Li, X. Yang, P. Sun, H. Qi and S. Lyu, ‘Celeb-DF: A large-scale challenging dataset for DeepFake forensics’, in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, Seattle, WA, USA, Jun. 2020, pp. 3207-3216.
- [6] B. Dolhansky, J. Bitton, B. Pflaum, J. Lu, R. Howes, M. Wang and C. C. Ferrer, ‘The DeepFake Detection Challenge (DFDC) dataset’, *arXiv preprint arXiv:2006.07397*, 2020.
- [7] Celeb-DF v2 dataset mirror on Kaggle, ‘reubensuju/celeb-df-v2’. Available: <https://www.kaggle.com/datasets/reubensuju/celeb-df-v2>
- [8] D. Afchar, V. Nozick, J. Yamagishi and I. Echizen, ‘MesoNet: A compact facial video forgery detection network’, in *Proc. IEEE Int. Workshop on Information Forensics and Security (WIFS)*, Hong Kong, Dec. 2018, pp. 1-7.
- [9] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li and L. Fei-Fei, ‘ImageNet: A large-scale hierarchical image database’, in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, Miami, FL, USA, Jun. 2009, pp. 248-255.
- [10] F. Chollet, ‘Xception: Deep learning with depthwise separable convolutions’, in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, Honolulu, HI, USA, Jul. 2017, pp. 1251-1258.
- [11] K. He, X. Zhang, S. Ren and J. Sun, ‘Deep residual learning for image recognition’, in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, Jun. 2016, pp. 770-778.
- [12] M. Tan and Q. V. Le, ‘EfficientNet: Rethinking model scaling for convolutional neural networks’, in *Proc. 36th Int. Conf. Machine Learning (ICML)*, Long Beach, CA, USA, Jun. 2019, pp. 6105-6114.
- [13] Q. Xie, M.-T. Luong, E. Hovy and Q. V. Le, ‘Self-training with Noisy Student improves ImageNet classification’, in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, Seattle, WA, USA, Jun. 2020, pp. 10687-10698.
- [14] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell and S. Xie, ‘A ConvNet for the 2020s’, in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, New Orleans, LA, USA, Jun. 2022, pp. 11976-11986.
- [15] Y. Li and S. Lyu, ‘Exposing DeepFake videos by detecting face warping artifacts’, in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition Workshops (CVPRW)*, Long Beach, CA, USA, Jun. 2019, pp. 46-52.

- [16] L. Li, J. Bao, T. Zhang, H. Yang, D. Chen, F. Wen and B. Guo, ‘Face X-ray for more general face forgery detection’, in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), Seattle, WA, USA, Jun. 2020, pp. 5001-5010.
- [17] H. Zhao, W. Zhou, D. Chen, T. Wei, W. Zhang and N. Yu, ‘Multi-attentional deepfake detection’, in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), Jun. 2021, pp. 2185-2194.
- [18] D. Güera and E. J. Delp, ‘Deepfake video detection using recurrent neural networks’, in Proc. 15th IEEE Int. Conf. Advanced Video and Signal Based Surveillance (AVSS), Auckland, New Zealand, Nov. 2018, pp. 1-6.
- [19] A. Haliassos, K. Vougioukas, S. Petridis and M. Pantic, ‘Lips don’t lie: A generalisable and robust approach to face forgery detection’, in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), Jun. 2021, pp. 5039-5049.
- [20] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit and N. Houlsby, ‘An image is worth 16x16 words: Transformers for image recognition at scale’, in Proc. Int. Conf. Learning Representations (ICLR), May 2021.
- [21] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin and B. Guo, ‘Swin Transformer: Hierarchical vision transformer using shifted windows’, in Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV), Montreal, QC, Canada, Oct. 2021, pp. 10012-10022.
- [22] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger and I. Sutskever, ‘Learning transferable visual models from natural language supervision’, in Proc. 38th Int. Conf. Machine Learning (ICML), Jul. 2021, pp. 8748-8763.
- [23] U. Ojha, Y. Li and Y. J. Lee, ‘Towards universal fake image detectors that generalize across generative models’, in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), Vancouver, BC, Canada, Jun. 2023, pp. 24480-24489.
- [24] P. Viola and M. Jones, ‘Rapid object detection using a boosted cascade of simple features’, in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), Kauai, HI, USA, Dec. 2001, pp. I-511-I-518.
- [25] K. Zhang, Z. Zhang, Z. Li and Y. Qiao, ‘Joint face detection and alignment using multitask cascaded convolutional networks’, IEEE Signal Processing Letters, Vol. 23, No. 10, pp. 1499-1503, Oct. 2016.
- [26] J. Deng, J. Guo, E. Ververas, I. Kotsia and S. Zafeiriou, ‘RetinaFace: Single-shot multi-level face localisation in the wild’, in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), Seattle, WA, USA, Jun. 2020, pp. 5203-5212.
- [27] J. Guo, J. Deng et al., ‘InsightFace: 2D and 3D face analysis project’, GitHub repository. Available: <https://github.com/deepsight/insightface>
- [28] G. Jocher, A. Chaurasia and J. Qiu, ‘Ultralytics YOLOv8’, software, version 8.0.0, 2023. Available: <https://github.com/ultralytics/ultralytics>
- [29] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh and D. Batra, ‘Grad-CAM: Visual explanations from deep networks via gradient-based localization’, in Proc. IEEE Int. Conf. Computer Vision (ICCV), Venice, Italy, Oct. 2017, pp. 618-626.
- [30] T. Fawcett, ‘An introduction to ROC analysis’, Pattern Recognition Letters, Vol. 27, No. 8, pp. 861-874, Jun. 2006.

- [31] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York; Chapman & Hall, 1993.
- [32] W. J. Youden, 'Index for rating diagnostic tests', *Cancer*, Vol. 3, No. 1, pp. 32-35, 1950.
- [33] C. Guo, G. Pleiss, Y. Sun and K. Q. Weinberger, 'On calibration of modern neural networks', in *Proc. 34th Int. Conf. Machine Learning (ICML)*, Sydney, Australia, Aug. 2017, pp. 1321-1330.
- [34] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan et al., 'PyTorch: An imperative style, high-performance deep learning library', in *Advances in Neural Information Processing Systems 32 (NeurIPS)*, Vancouver, Canada, Dec. 2019, pp. 8024-8035.
- [35] R. Wightman, 'PyTorch Image Models (timm)', GitHub repository, 2019. Available: <https://github.com/huggingface/pytorch-image-models>
- [36] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel et al., 'Scikit-learn: Machine learning in Python', *Journal of Machine Learning Research*, Vol. 12, pp. 2825-2830, 2011.
- [37] I. Loshchilov and F. Hutter, 'Decoupled weight decay regularization', in *Proc. Int. Conf. Learning Representations (ICLR)*, New Orleans, LA, USA, May 2019.
- [38] I. Loshchilov and F. Hutter, 'SGDR: Stochastic gradient descent with warm restarts', in *Proc. Int. Conf. Learning Representations (ICLR)*, Toulon, France, Apr. 2017.
- [39] P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh and H. Wu, 'Mixed precision training', in *Proc. Int. Conf. Learning Representations (ICLR)*, Vancouver, BC, Canada, May 2018.
- [40] K. Zuiderveld, 'Contrast limited adaptive histogram equalization', in *Graphics Gems IV*, P. S. Heckbert, Editor. San Diego, CA; Academic Press, 1994, pp. 474-485.

Appendices

Appendix A lists the configuration of the final notebook, Appendix B maps the sections and cells of both notebooks to their purpose and output, and Appendix C lists the files produced by the pipeline.

Appendix A: Configuration of the Final Notebook

Table A.1: Configuration values of the final notebook (Run A)

Parameter	Value
kaggle_dataset	reubensuju/celeb-df-v2 (auto-discovered under /kaggle/input)
real_dirs / fake_dirs	Celeb-real, YouTube-real / Celeb-synthesis
test_list	List_of_testing_videos.txt
val_frac / holdout_frac / split_seed	0.15 / 0.10 / 42
frames_real / frames_fake / frames_test	32 / 8 / 16
edge_trim	0.05
det_size / min_face_score	640 / 0.5
align_size / jpeg_quality / cache_shards	256 / 95 / 12
img_size / batch_size / num_workers	224 / 64 / 2
epochs / freeze_epochs / patience	8 / 1 / 3
lr_head	1e-3
lr_backbone, weight_decay, drop_path_rate	Per model (Table 5.3)
warmup_frac / grad_clip / drop_rate	0.05 / 1.0 / 0.3
SEED	42
Robustness faces per video	Up to 8
Bootstrap resamples	1,000 (stratified)
Temperature grid	300 values, log-spaced 0.1-100
Demo / resolution-check frames	32 / 32

Run B uses the same core data and ConvNeXt-Tiny settings as Run A, plus 64 comparison frames and a 360-pixel minimum height for the enhancement step. Its threshold is selected only from validation videos.

Appendix B: Notebook Maps

Table B.1: Sections of the final notebook (Run A)

Section	Purpose	Key output
1	Setup: packages, imports, seeds, configuration, model registry, dataset discovery	torch 2.11.0, Tesla T4
2	Data: counts, identity parsing, split, leakage audit, demo videos	Audit passed; 36 shared test identities
3	Face extraction: RetinaFace, frame reading, alignment, face cache	73,429 crops
4	Training components: augmentation, dataset, model, optimiser, metrics, trainer	pos_weight = 0.508
5	Train all models (seed 42)	7 model runs completed; ConvNeXt-Tiny selected
6	Comparison: bootstrap CI, ROC, confusion matrices, curves, speed	ConvNeXt-Tiny holdout AUC 0.9998; official AUC 0.9990
7	Robustness: JPEG, downscale, blur	Worst drop 0.0010 (ConvNeXt-Tiny)
8	Calibration: temperature scaling, log loss, ECE	No video-level gain
9	Explainability: Grad-CAM for every model	1 real + 1 fake face
10	End-to-end demo and export	2 / 2 demo videos correct; artifacts saved
11	High vs low resolution check (four videos)	4 / 4 correct

Table B.2: Cells of the final Kaggle notebook (Run B)

Cell	Purpose	Key output
GPU check	nvidia-smi; fails if no GPU	Tesla T4 detected
Cell 0	PyTorch / CUDA check	PyTorch CUDA available; Tesla T4
Cell 1	Kaggle-safe dependency setup	Dependency setup passed
Cell 2	Imports, seeds, GPU / ORT pre-flight	Environment preflight passed
Cell 3	Configuration dataclass and paths	Checkpoint / metrics paths
Cell 4	Choose input option 1 / 2 / 3; validate dataset	Option 2; 590 / 300 / 5,639 videos
Cell 5	Manifest, identity split, leakage audit	Audit passed; split table
Cell 6	RetinaFace, frame sampling, eye alignment	Pipeline ready
Cell 7	Build / restore face cache (12 shards)	73,429 indexed aligned face crops
Helper	Copy attached best.pt if present	None attached; train
Cell 8	Dataset, augmentation, model, loaders	pos_weight = 0.5079
Cell 9	Metrics, optimiser, scheduler, training helpers	Ready
Cell 10	Train or load ConvNeXt-Tiny	8 epochs; epoch 5 selected
Cell 11	Youden threshold; validation / holdout metrics	t = 0.4825; 98.78% / 97.28%
Cell 12	Low-resolution frame enhancement	Ready
Cell 13	Select low-resolution test videos	3 REAL videos selected
Cell 14	Show original vs enhanced face crops	6 figures
Cell 15	Per-video inference function	Ready
Cell 16	Original vs enhanced test and summary	3 / 3 correct; no decision change

Appendix C: Files Produced by the Pipeline

Table C.1: Output files (under the retinaface_models working folder)

File	Content
videos.csv	Manifest: path, folder, label, identities, component, split
face_cache/shard_00.tar ... shard_11.tar	Aligned face crops (JPEG, quality 95)
runs/<model>_s42/best.pt	Best weights of each model
runs/<model>_s42/history.csv	Per-epoch loss, validation AUC and EER
runs/<model>_s42/preds_{val,holdout,test}.csv	Frame-level predictions
runs/<model>_s42/metrics.json	Threshold, best epoch, speed and all split metrics
runs/<model>_s42/robustness.csv	AUC under each degradation
demo_videos/	First holdout video of each class
deepfake_convnext_tiny_final.pt	Inference bundle: weights, threshold, pre-processing settings
model_comparison.csv	Summary table of the seven models
participant4_resolution_4video_results.csv	High vs low resolution check
runs/convnext_tiny_s42/{best.pt, history.csv, metrics.json}	Run B independent ConvNeXt-Tiny checkpoint and metrics

List of Publications / Conference Presentations

No publications or conference presentations have resulted from this work so far. The results were presented in the capstone project presentation “Deepfake Detection – Low-Resolution & 64-Frame Evaluation”.

Checklist of Items for the Final Report

#	Item	Y / N
a)	Is the Cover page in proper format?	Y
b)	Is the Title page in proper format?	Y
c)	Is the Certificate from the Mentor in proper format? Has it been signed?	Y / N (to be signed)
d)	Is Abstract included in the Report? Is it properly written?	Y
e)	Does the Table of Contents page include chapter page numbers?	Y
f)	Does the Report contain a summary of the literature survey?	Y
i.	Are the Pages numbered properly?	Y
ii.	Are the Figures numbered properly?	Y
iii.	Are the Tables numbered properly?	Y
iv.	Are the Captions for the Figures and Tables proper?	Y
v.	Are the Appendices numbered?	Y
g)	Does the Report have Conclusion / Recommendations of the work?	Y
h)	Are References/Bibliography given in the Report?	Y
i)	Have the References been cited in the Report?	Y
j)	Is the citation of References / Bibliography in proper format?	Y

Note: A copy of this checklist is included as the last page of the Final Report. This checklist, duly completed and signed by the students, should also be verified and signed by the evaluators.

Signature of the Students:

1. _____

2. _____

3. _____

Signature of the Evaluators:

1. _____

2. _____